ÉCOLE NATIONALE D'INGÉNIEURS DE CARTHAGE
ENICarthage — Université de Carthage

COMPTE RENDU

Ateliers de test logiciel

Recette de la plateforme **EniGov**
Plateforme numérique du délégué étudiant

Réalisé par : **Mahmoud Amri**
Encadrement : **Faouzi Jaidi**
Outil de gestion des tests : Squash TM 15.0.3
Date : 4 novembre 2026

Année universitaire 2026 – 2027

Sommaire

1. Introduction et objectifs des ateliers
2. Choix du projet : EniGov (objectif 1)
3. Installation de l'environnement de test
4. Besoins fonctionnels et non fonctionnels (objectif 2)
5. Outil de gestion des tests et des exigences : Squash TM (objectif 3)
6. Plan de test (objectif 4)
7. Exécution du plan de test (objectif 5)
    1. Tests statiques
    2. Tests unitaires
    3. Tests d'intégration
    4. Tests structurels (couverture)
    5. Tests système (E2E)
    6. Tests d'acceptation manuels
    7. Tests non fonctionnels
    8. Tests liés aux changements et de maintenance
    9. Suivi de la campagne dans Squash TM
8. Bilan : anomalies, métriques et verdict de recette
9. Conclusion
10. Annexe : rejouer les tests

1. Introduction et objectifs des ateliers

Ce compte rendu présente le travail réalisé dans le cadre des ateliers de test logiciel. Il suit les cinq objectifs fixés :

Objectif	Travail réalisé	Chapitre
1. Choisir un projet	Plateforme web EniGov (Spring Boot + React + PostgreSQL), clonée et lancée en local	2, 3
2. Présenter les besoins fonctionnels et non fonctionnels	18 exigences fonctionnelles et 9 non fonctionnelles, avec criticité	4
3. Utiliser un outil de gestion de test (et des exigences)	Squash TM 15.0.3 installé sous Docker : exigences, cas de test, traçabilité, campagne	5
4. Élaborer un plan de test	Plan couvrant 4 niveaux (unitaire, intégration, système, acceptation) et 5 types (fonctionnel, non fonctionnel, structurel, lié aux changements, maintenance)	6
5. Exécuter le plan de test	Techniques statiques et dynamiques, tests manuels et automatisés ; 8 anomalies détectées dont 1 critique corrigée et re-testée	7, 8

27
exigences

33
cas de test dans Squash TM

52 + 7 + 14
tests JUnit · E2E · étapes manuelles

8
anomalies détectées

2. Choix du projet : EniGov (objectif 1)

2.1 Présentation

EniGov est la plateforme numérique de gouvernance étudiante de l'ENICarthage (dépôt : `github.com/Mahmoud35amri/EniGov`). Le délégué élu y publie des annonces, gère les réclamations des étudiants, organise des sondages, publie des décisions et des événements, partage des documents et échange par messagerie. Les étudiants, inscrits avec une adresse `@enicar.ucar.tn`, consultent ces informations, déposent des réclamations (privées ou publiques), votent et écrivent au délégué.

2.2 Pourquoi ce projet ?

- **Application réelle et complète** : backend Java, frontend React, base de données, authentification ; tous les niveaux de test s'y appliquent.
- **Deux rôles** (DÉLÉGUÉ, ÉTUDIANT) avec des règles d'accès : terrain idéal pour les tests de sécurité et d'autorisation.
- **Règles métier testables** : vote unique, ciblage des sondages, verrouillage après 5 échecs, politique de mot de passe (valeurs limites, tables de décision).
- **Données sensibles** : réclamations privées, d'où des exigences de confidentialité à vérifier.

2.3 Architecture technique

Couche	Technologies
Frontend	React 18, Vite 5, React Router 6, Tailwind CSS, Axios, Framer Motion
Backend	Java 17 (exécuté sur JDK 25), Spring Boot 3.2.2, Spring Security + JWT (jjwt 0.11.5), Spring Data JPA / Hibernate
Base de données	PostgreSQL (schéma généré par Hibernate : `ddl-auto=update`)
Services externes	Cloudinary (fichiers), Brevo (e-mails) — non configurés en local

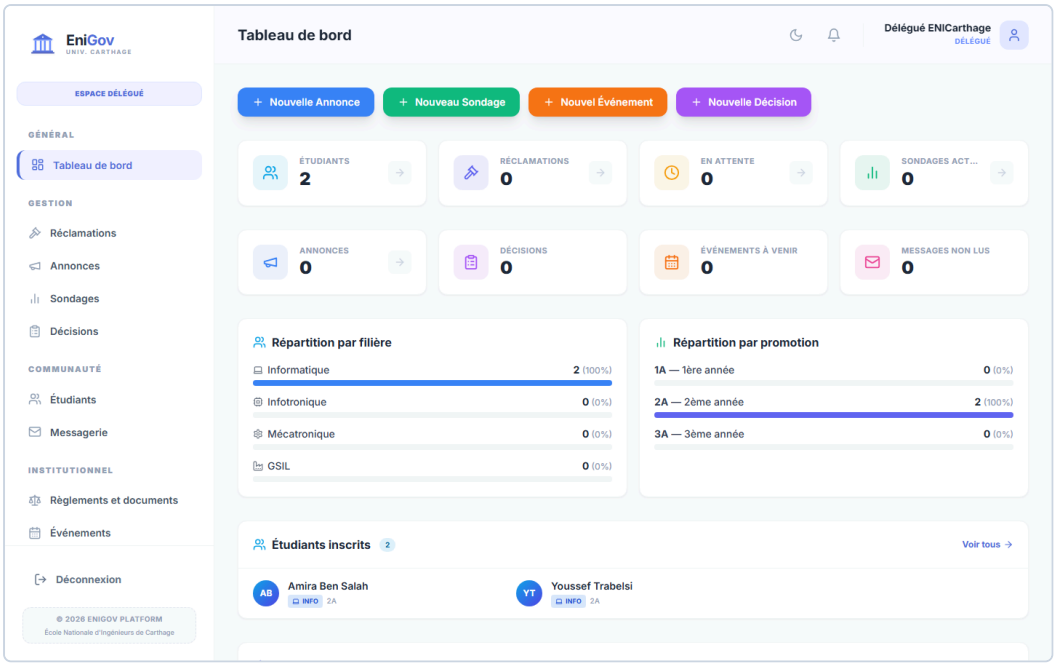


Figure 1 – Tableau de bord du délégué (application lancée en local)

3. Installation de l'environnement de test

3.1 Outils installés

Outil	Version	Rôle	Installation
Git	—	Clonage du dépôt	`git clone https://github.com/Mahmoud35amri/EniGov.git`
JDK / Maven	JDK 25 · Maven 3.9.12	Build et tests backend	déjà présents
Node.js / npm	22.23 · 12.2	Build frontend, Playwright	déjà présents ; `npm install`
Docker Desktop	29.1	Conteneurs PostgreSQL, Squash TM, k6	déjà présent
PostgreSQL	16 (conteneur)	Base de l'application, base de test, base de Squash	`docker run postgres:16 -p 5433:5432`
Squash TM	**15.0.3**	**Gestion des exigences et des tests**	`docker run squashtest/squash -p 8090:8080`
JUnit 5, Mockito, MockMvc	Spring Boot Test 3.2	Tests unitaires et d'intégration	déjà dans le `pom.xml`
JaCoCo	0.8.14	Couverture de code (tests structurels)	plugin ajouté au `pom.xml`
PMD / CPD	7.17	Analyse statique Java, détection de duplications	`mvn pmd:pmd pmd:cpd`
ESLint	8 + plugins React	Analyse statique JavaScript	`npm i -D eslint@8 …` + `.eslintrc.cjs`
Playwright	1.x + Chromium	Tests système E2E, aide à l'exécution des tests manuels	`npm i -D @playwright/test`
k6	grafana/k6 (Docker)	Test de charge	`docker run grafana/k6`

Environnement : versions des outils et conteneurs Docker

```
$ java -version
java version "25" 2025-09-16 LTS
$ mvn -v
Apache Maven 3.9.12 (848fbb4bf2d427b72bdb2471c22fced7ebd9a7a
$ node -v && npm -v
v22.23.4  /  npm 12.2.1
$ docker ps --format "table {{.Names}}  {{.Image}}      {{.Status}}     {{.Ports}}"
NAMES             IMAGE                     STATUS             PORTS
squash-tm         squashtest/squash:latest  Up 9 minutes       0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp
enigov-postgres   postgres:16               Up About an hour   0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
```

Figure 2 – Versions des outils et conteneurs en cours d'exécution

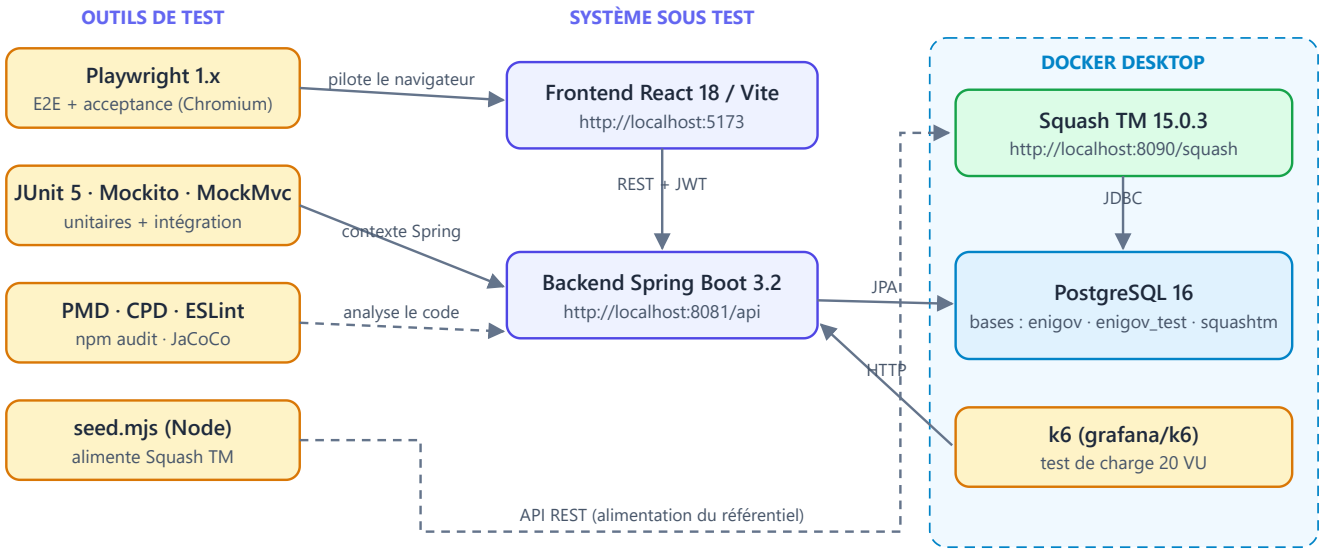


Figure 3 – Architecture de l'environnement de test

3.2 Problèmes rencontrés et solutions

Problème	Solution
Le PostgreSQL local (port 5432) n'utilise pas le mot de passe attendu par le projet	PostgreSQL 16 dédié dans Docker, exposé sur le port 5433
Le backend refuse de démarrer : variables `CLOUDINARY_*` obligatoires mais absentes de la documentation	Valeurs factices dans `backend/.env` (anomalie de documentation **ANO-08**)
Le port 8080 est occupé par un service Oracle (TNSLSNR)	Squash TM exposé sur le port 8090
Squash TM 15 ne démarre pas sans base externe (H2 n'est plus embarquée)	Base `squashtm` créée dans le conteneur PostgreSQL ; variables `SQTM_DB_*`
L'API REST de Squash refuse l'authentification Basic par défaut	Propriété `squash.rest-api.disallow-basic-authentication=false` (variable d'environnement)
Mockito / Byte Buddy ne supporte pas officiellement le JDK 25	Option `-Dnet.bytebuddy.experimental=true` dans le `pom.xml` (voir test de maintenance TM-01)

4. Besoins fonctionnels et non fonctionnels (objectif 2)

Les besoins ont été extraits de la documentation du projet (README) et vérifiés par la lecture du code. Chaque exigence porte une référence unique, reprise dans Squash TM pour assurer la traçabilité vers les cas de test. La criticité oriente l'effort de test (approche basée sur les risques).

4.1 Acteurs

Acteur	Description
Délégué	Compte unique pré-créé ; administre la plateforme (publication, traitement des réclamations, statistiques).
Étudiant	Inscrit avec un e-mail @enicar.ucar.tn vérifié ; consulte, dépose des réclamations, vote, écrit au délégué.

4.2 Exigences fonctionnelles

Réf.	Exigence	Description	Catégorie	Criticité
EF-01	Inscription des étudiants	Un étudiant s'inscrit avec un e-mail @enicar.ucar.tn unique, son nom, sa filière et sa promotion. Le mot de passe contient 8 à 40 caractères, dont au moins une majuscule et un chiffre.	Fonctionnelle	Critique
EF-02	Vérification de l'adresse e-mail	Un compte n'est utilisable qu'après le clic sur le lien de vérification reçu par e-mail ; avant cela, la connexion est refusée.	Fonctionnelle	Majeure
EF-03	Connexion sécurisée et verrouillage du compte	Connexion par e-mail et mot de passe délivrant un jeton JWT. Après 5 échecs consécutifs, le compte est verrouillé pendant 15 minutes.	Sécurité	Critique
EF-04	Mot de passe oublié	Réinitialisation du mot de passe via un lien envoyé par e-mail, valable 15 minutes.	Fonctionnelle	Mineure
EF-05	Dépôt de réclamations	L'étudiant dépose une réclamation (titre, description, pièce jointe optionnelle), privée ou publique. Le statut initial est « En attente ».	Fonctionnelle	Critique
EF-06	Confidentialité des réclamations privées	Une réclamation privée n'est visible que par l'étudiant qui l'a créée et par le délégué.	Sécurité	Critique
EF-07	Vote sur les réclamations publiques	Les étudiants votent pour ou contre une réclamation publique (bascule, un seul vote par étudiant). Score = pour − contre. Vote impossible sur une réclamation privée.	Fonctionnelle	Majeure
EF-08	Traitement des réclamations par le délégué	Le délégué change le statut (En cours, Résolue, Rejetée) et répond ; l'étudiant concerné est notifié.	Fonctionnelle	Critique
EF-09	Publication des annonces	Le délégué publie des annonces (titre, contenu, pièce jointe) ; les étudiants les consultent en lecture seule.	Fonctionnelle	Majeure
EF-10	Création et ciblage des sondages	Le délégué crée un sondage (au moins 2 options, date limite optionnelle, ciblage par filière et promotion) et peut le clôturer.	Fonctionnelle	Majeure
EF-11	Vote aux sondages	Un étudiant vote une seule fois, sans modification possible, tant que le sondage est ouvert et avant la date limite. Les résultats s'affichent en pourcentages.	Fonctionnelle	Critique
EF-12	Décisions	Le délégué publie des décisions liées à une réclamation ou à un sondage.	Fonctionnelle	Mineure
EF-13	Messagerie étudiant ↔ délégué	Un étudiant ne peut écrire qu'au délégué ; le délégué peut écrire à tout étudiant. Statut lu / non lu.	Fonctionnelle	Majeure

Réf.	Exigence	Description	Catégorie	Criticité
EF-14	Calendrier des événements	Le délégué crée et supprime des événements ; les étudiants consultent le calendrier.	Fonctionnelle	Mineure
EF-15	Règlements et documents	Le délégué dépose des PDF ; les étudiants les consultent et les téléchargent.	Fonctionnelle	Mineure
EF-16	Notifications in-app	Cloche avec compteur de notifications non lues, déclenchées par : annonce, changement de statut de réclamation, sondage, décision, message, événement.	Fonctionnelle	Majeure
EF-17	Tableau de bord du délégué	Statistiques agrégées (étudiants, réclamations par statut, sondages, annonces…) réservées au délégué.	Fonctionnelle	Mineure
EF-18	Rôles et contrôle d'accès	Deux rôles : DÉLÉGUÉ et ÉTUDIANT. Les fonctions de gestion (publication, traitement, statistiques, liste des étudiants) sont réservées au délégué.	Sécurité	Critique

4.3 Exigences non fonctionnelles

Réf.	Exigence	Description	Catégorie	Criticité
ENF-01	Sécurité des identifiants et des jetons	Mots de passe hachés avec BCrypt ; API sans état authentifiée par JWT signé (HS256). Un jeton falsifié ou expiré est rejeté.	Sécurité	Critique
ENF-02	Protection de l'API	Toute requête non authentifiée ou non autorisée est refusée (HTTP 401/403), sans erreur serveur (500).	Sécurité	Critique
ENF-03	Performance	95 % des requêtes de consultation répondent en moins de 500 ms avec 20 utilisateurs simultanés.	Performance	Majeure
ENF-04	Fiabilité sous charge	Taux d'erreur inférieur à 1 % sous la charge nominale (20 utilisateurs).	Performance	Majeure
ENF-05	Ergonomie et affichage responsive	Interface utilisable sur smartphone (largeur ≥ 360 px) sans défilement horizontal ; thème clair/sombre mémorisé.	Ergonomie	Majeure
ENF-06	Maintenabilité	Code analysé statiquement (PMD, CPD, ESLint) ; couverture des tests automatisés ≥ 40 % des instructions du backend.	Technique	Mineure
ENF-07	Portabilité	Fonctionne sur JDK 17+ et Node.js 18+ ; déployable par conteneur (Dockerfile, Render, Vercel).	Technique	Mineure
ENF-08	Sécurité des dépendances	Aucune dépendance présentant une vulnérabilité connue de sévérité haute ou critique.	Sécurité	Majeure
ENF-09	Intégrité et persistance des données	Les données saisies (base PostgreSQL) sont conservées après un redémarrage de l'application.	Technique	Critique

5. Outil de gestion des tests et des exigences : Squash TM (objectif 3)

5.1 Choix de l'outil

Squash TM est un outil libre de gestion des tests qui gère dans un même référentiel les **exigences**, les **cas de test** et les **campagnes d'exécution**, avec la traçabilité entre eux. Il répond donc à la consigne : simplifier la gestion des tests tout en gérant les exigences du logiciel. Il est en français, gratuit en version Community et s'installe sous Docker.

5.2 Organisation du projet EniGov dans Squash TM

- **Exigences** : 27 exigences réparties en deux dossiers (fonctionnelles / non fonctionnelles), avec catégorie, criticité et statut « Approuvée ».
- **Cas de test** : 33 cas répartis en six dossiers : « 01 - Tests unitaires (JUnit 5 + Mockito) », « 02 - Tests d'intégration (Spring Boot + MockMvc + PostgreSQL) », « 03 - Tests système (E2E Playwright) », « 04 - Tests d'acceptation (manuels) », « 05 - Tests non fonctionnels », « 06 - Tests liés aux changements et de maintenance ». Chaque cas a une nature, un type, une importance, des pas de test (action → résultat attendu) et un lien vers les exigences qu'il vérifie.
- **Campagne** « Campagne de recette EniGov v1.0 » avec deux itérations : build initial v1.0.0, puis build corrigé v1.0.1.

Pour éviter une saisie manuelle de plus de 30 cas de test, le référentiel a été écrit une seule fois (`qa/squash/referentiel.mjs`), puis injecté dans Squash TM par son **API REST** (script `qa/squash/seed.mjs`). Le même fichier sert à générer les tableaux de ce compte rendu, ce qui garantit leur cohérence.



Figure 4 – Alimentation de Squash TM par l'API REST

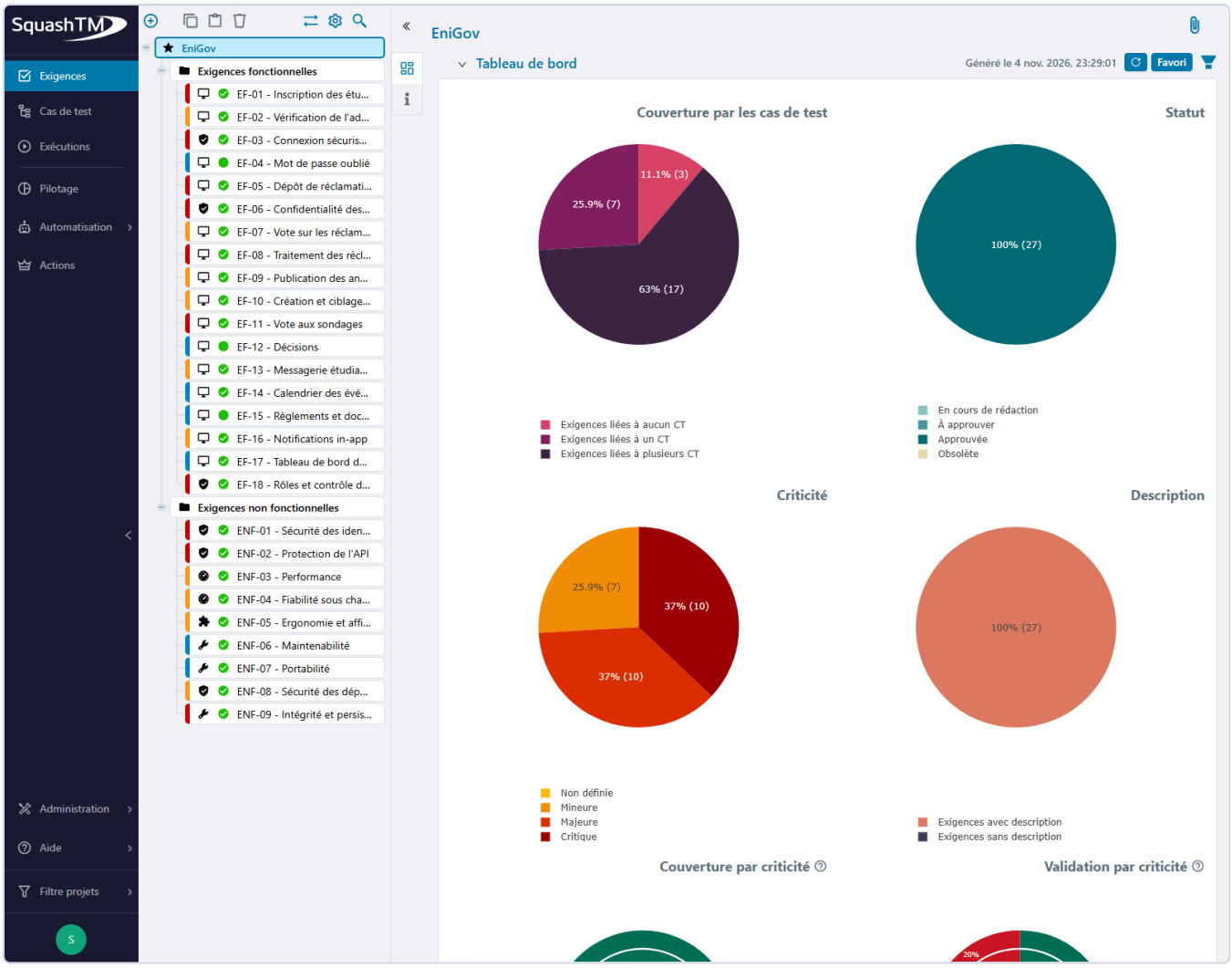


Figure 5 – Espace « Exigences » : arborescence et tableau de bord du projet (couverture, statut, criticité)

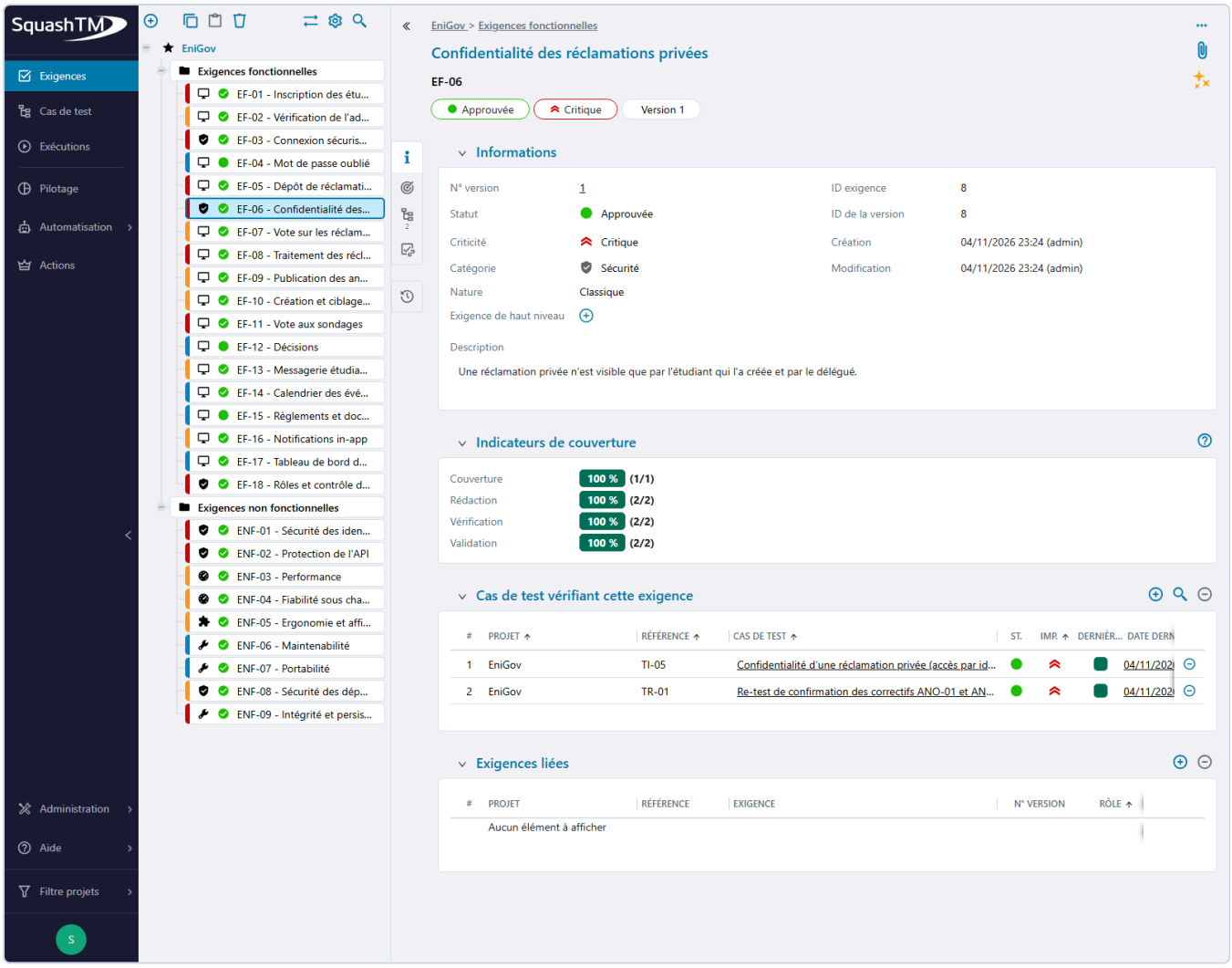


Figure 6 – Exigence EF-06 : criticité, catégorie, indicateurs de couverture et cas de test qui la vérifient

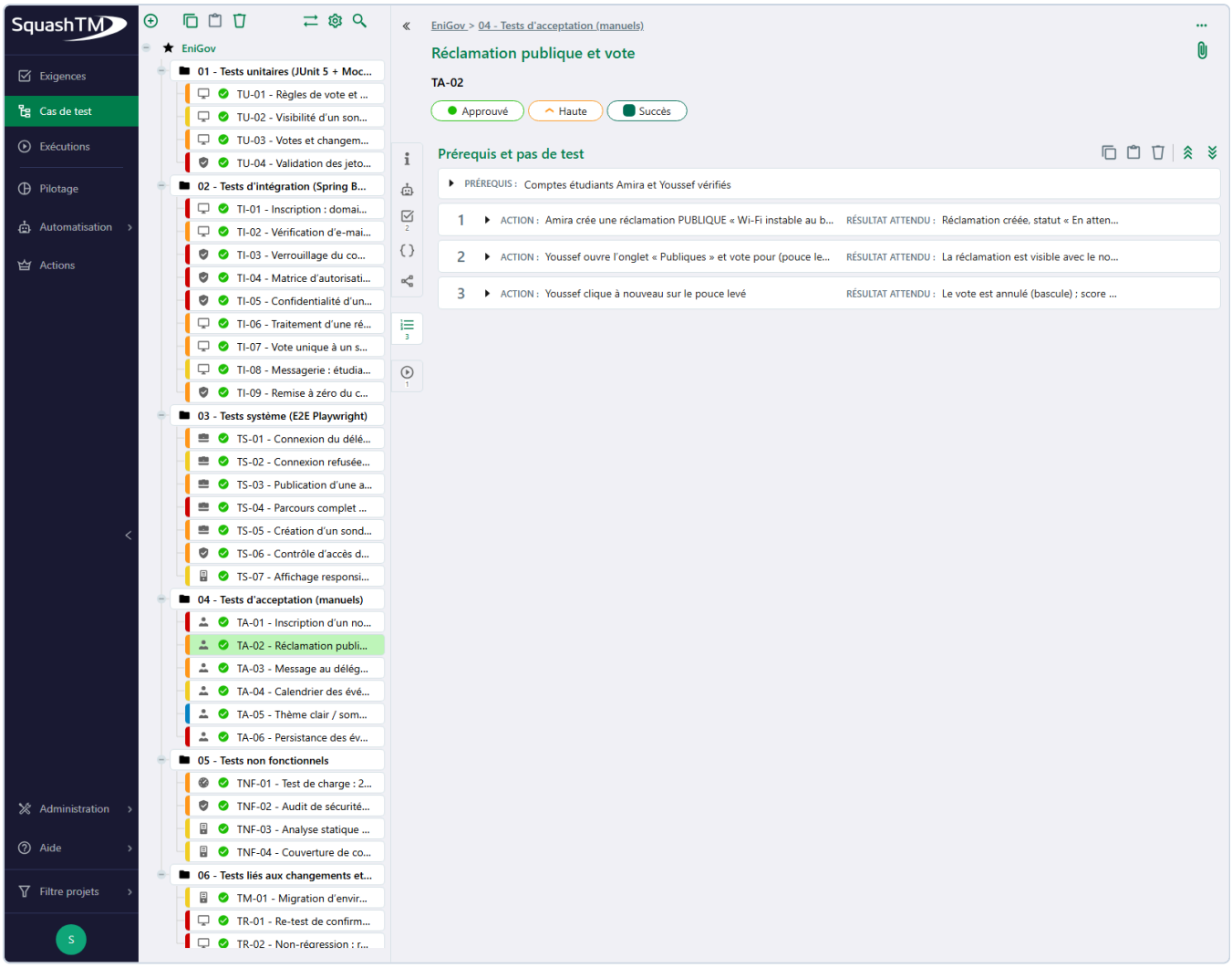


Figure 7 – Espace « Cas de test » : arborescence par niveau de test et pas de test du cas TA-02

5.3 Matrice de traçabilité exigences → cas de test

Exigence	Criticité	Cas de test qui la vérifient
EF-01 Inscription des étudiants	Critique	TI-01, TA-01, TR-02
EF-02 Vérification de l'adresse e-mail	Majeure	TI-02, TA-01
EF-03 Connexion sécurisée et verrouillage du compte	Critique	TU-04, TI-03, TI-09, TS-01, TS-02, TR-02
EF-04 Mot de passe oublié	Mineure	**non couverte**
EF-05 Dépôt de réclamations	Critique	TU-03, TS-04, TA-02, TR-02
EF-06 Confidentialité des réclamations privées	Critique	TI-05, TR-01
EF-07 Vote sur les réclamations publiques	Majeure	TU-03, TA-02
EF-08 Traitement des réclamations par le délégué	Critique	TU-03, TI-06, TS-04
EF-09 Publication des annonces	Majeure	TS-03
EF-10 Création et ciblage des sondages	Majeure	TU-01, TU-02, TS-05
EF-11 Vote aux sondages	Critique	TU-01, TI-07, TS-05, TR-02
EF-12 Décisions	Mineure	**non couverte**
EF-13 Messagerie étudiant ↔ délégué	Majeure	TI-08, TA-03

Exigence	Criticité	Cas de test qui la vérifient
EF-14 Calendrier des événements	Mineure	TA-04, TA-06
EF-15 Règlements et documents	Mineure	non couverte
EF-16 Notifications in-app	Majeure	TI-06, TA-03
EF-17 Tableau de bord du délégué	Mineure	TI-04, TS-01
EF-18 Rôles et contrôle d'accès	Critique	TI-04, TS-06, TR-02
ENF-01 Sécurité des identifiants et des jetons	Critique	TU-04, TR-01
ENF-02 Protection de l'API	Critique	TI-04
ENF-03 Performance	Majeure	TNF-01
ENF-04 Fiabilité sous charge	Majeure	TNF-01
ENF-05 Ergonomie et affichage responsive	Majeure	TS-07, TA-05
ENF-06 Maintenabilité	Mineure	TNF-03, TNF-04
ENF-07 Portabilité	Mineure	TM-01
ENF-08 Sécurité des dépendances	Majeure	TNF-02
ENF-09 Intégrité et persistance des données	Critique	TA-06

Couverture des exigences : 24/27 (89 %). Les exigences non couvertes (EF-04, EF-12, EF-15) ont été exclues volontairement : EF-04 et EF-15 dépendent de services externes (envoi d'e-mails Brevo, stockage Cloudinary) non configurés en local, et EF-12 est de criticité mineure. Ce choix est expliqué dans le plan de test (§ 6.1).

6. Plan de test (objectif 4)

Ce plan s'inspire de la structure de la norme IEEE 829 / ISO 29119-3.

6.1 Objet et périmètre

Dans le périmètre	Hors périmètre (justification)
• Authentification, inscription, verrouillage, rôles • Réclamations, votes, traitement, notifications • Sondages, annonces, événements, messagerie • Sécurité de l'API, performance, responsive, maintenabilité, dépendances, persistance	• EF-04 Mot de passe oublié : l'envoi réel d'e-mail (Brevo) n'est pas configuré • EF-15 Documents : l'upload passe par Cloudinary, non configuré • EF-12 Décisions : criticité mineure, reportée • Déploiement cloud (Render / Vercel)

6.2 Stratégie : approche basée sur les risques

Risque produit	Probabilité	Impact	Priorité	Tests prévus
Fuite de réclamations privées	Moyenne	Critique	P1	Revue de code, TI-05, TR-01
Accès non autorisé aux fonctions du délégué	Moyenne	Critique	P1	TI-04, TS-06, TU-04
Fraude au vote (double vote)	Moyenne	Majeur	P1	TU-01, TI-07, TS-05
Comptes piratés (force brute)	Faible	Critique	P2	TI-03, TI-09
Perte de données	Faible	Critique	P2	Revue de code, TA-06
Lenteur sous charge	Faible	Majeur	P2	TNF-01
Mauvais affichage sur mobile	Moyenne	Moyen	P2	TS-07, TA-05
Dépendances vulnérables	Élevée	Majeur	P2	TNF-02

6.3 Niveaux et types de test

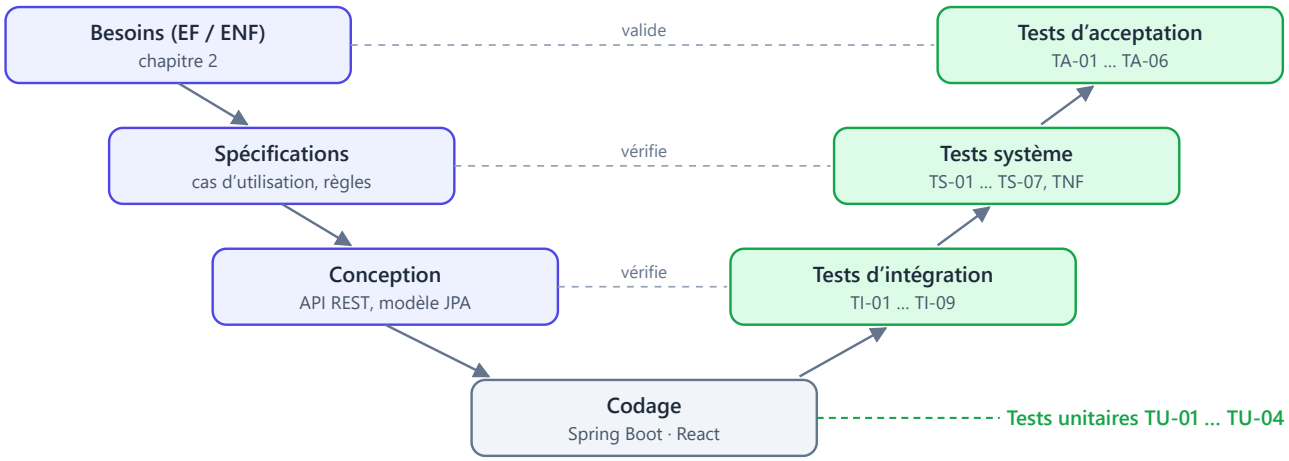


Figure 8 – Correspondance entre les phases de développement et les niveaux de test (cycle en V)

Niveau \ Type	Fonctionnel	Non fonctionnel	Structurel (boîte blanche)	Lié aux changements	Maintenance
Unitaire	TU-01, TU-02, TU-03	TU-04 (sécurité)	TNF-04 (JaCoCo)	TR-01 (re-test TU-04)	—
Intégration	TI-01, 02, 06, 07, 08	TI-03, TI-04, TI-09 (sécurité)	TNF-04 (JaCoCo)	TR-01 (re-test TI-05), TR-02	TM-01 (JDK 25)
Système	TS-01 … TS-06	TS-07 (responsive), TNF-01 (charge), TNF-02 (dépendances)	—	TR-02 (régression E2E)	TM-01 (PostgreSQL 16 / Docker)
Acceptation	TA-01 … TA-04	TA-05 (utilisabilité)	—	—	TA-06 (persistance au redémarrage)

6.4 Techniques de test

Famille	Technique	Application dans le projet
Statique (sans exécuter le code)	Revue de code manuelle	Lecture des contrôleurs et services : 3 anomalies trouvées avant tout test dynamique (ANO-01, 03, 04)
	Analyse statique outillée	PMD et CPD (Java), ESLint (JavaScript)
	Analyse de composition logicielle	`npm audit` sur les dépendances du frontend
	Revue de la documentation	README / guide de déploiement confrontés à l'installation réelle (ANO-08)
Dynamique boîte noire	Partitions d'équivalence	Mots de passe valides / invalides, jetons JWT (TU-04, TI-01)
	Analyse des valeurs limites	Mot de passe 7/8 et 40/41 caractères ; 1 et 2 options ; index −1 et 3 ; 4e et 5e échec de connexion
	Table de décision	Visibilité d'un sondage selon la filière et la promotion (TU-02, 9 règles)
	Transitions d'états	Vote pour / contre / annulé ; statut de réclamation ; verrouillage du compte
	Cas d'utilisation	Scénarios E2E et d'acceptation multi-acteurs
Dynamique boîte blanche	Couverture des instructions et des branches	JaCoCo sur l'ensemble des tests JUnit (TNF-04)
Basée sur l'expérience	Test exploratoire guidé par la revue	TA-06 : vérifier la persistance des événements après redémarrage

6.5 Tests manuels et automatisés

Mode	Cas de test	Outils
Automatisé	TU-01, TU-02, TU-03, TU-04, TI-01, TI-02, TI-03, TI-04, TI-05, TI-06, TI-07, TI-08, TI-09, TS-01, TS-02, TS-03, TS-04, TS-05, TS-06, TS-07, TNF-01, TNF-02, TNF-03, TNF-04, TR-01, TR-02, TM-01	JUnit 5, Mockito, MockMvc, Playwright, k6, PMD, ESLint, npm audit, JaCoCo
Manuel	TA-01, TA-02, TA-03, TA-04, TA-05, TA-06	Navigateur, fiches de test Squash TM, terminal (curl)

6.6 Environnement et données de test

- Tests d'intégration : base dédiée `enigov_test`, recréée à chaque exécution (`ddl-auto=create-drop`, profil Spring `test`). Seul l'envoi d'e-mails est simulé (`@MockBean EmailService`).
- Tests système et d'acceptation : base `enigov` ; deux étudiants fictifs (Amira Ben Salah, Youssef Trabelsi) créés par l'API et validés en base au début de chaque exécution (`global-setup.js`) ; compte délégué pré-créé.
- Navigateur : Chromium (bureau 1440 × 900 et émulation Pixel 7).

6.7 Critères d'entrée, de sortie et de suspension

Critère	Définition
Entrée	Application compilée et démarrée ; base de test accessible ; exigences approuvées dans Squash TM
Sortie	100 % des cas planifiés exécutés ; aucune anomalie critique ouverte ; couverture des exigences ≥ 85 % ; couverture des instructions backend ≥ 40 %
Suspension	Environnement indisponible ou anomalie bloquante empêchant plus de 30 % des tests
Reprise	Correctif livré ; re-test de confirmation puis non-régression

6.8 Gestion des anomalies

Cycle de vie : **Détectée** → **Confirmée** → **Corrigée** → **Re-testée** → **Fermée** (ou **Ouverte** si le correctif est planifié plus tard). Gravité : **Critique** (sécurité ou perte de données, bloque la mise en production), **Majeure** (fonction dégradée), **Mineure** (gêne, qualité). Chaque anomalie est référencée (ANO-xx) dans le commentaire de l'exécution Squash TM correspondante.

6.9 Planning et livrables

Phase	Livrables
1. Installation et lancement	Environnement Docker, application démarrée
2. Analyse des besoins	Exigences EF / ENF dans Squash TM
3. Conception des tests	Cas de test, matrice de traçabilité, plan de test
4. Itération 1 (build v1.0.0)	Exécution des tests unitaires et d'intégration, anomalies
5. Correctifs	Correction de ANO-01 et ANO-02
6. Itération 2 (build v1.0.1)	Re-test, non-régression, tests système, acceptation, non fonctionnels, maintenance
7. Clôture	Bilan, compte rendu

7. Exécution du plan de test (objectif 5)

7.1 Tests statiques

7.1.1 Revue de code manuelle

Avant toute exécution, les contrôleurs et services du backend ont été relus en les confrontant aux exigences. Trois défauts ont été identifiés, puis confirmés par des tests dynamiques :

```
// ComplaintController.java — aucune vérification du propriétaire (EF-06)  → ANO-01
@GetMapping("/{id}")
public ResponseEntity<ComplaintResponse> getComplaint(@PathVariable String id, Principal principal) {
    return ResponseEntity.ok(complaintService.getById(id, principal.getName()));   // tout étudiant
authentifié
}

// DatabaseSeeder.java — exécuté à CHAQUE démarrage (ENF-09)                → ANO-04
public void run(String... args) {
    eventRepository.deleteAll();     // supprime aussi les événements créés par le délégué
    seedEvents();
}

// AuthController.java — le compteur n'est remis à 0 qu'après une connexion réussie (EF-03) → ANO-03
int attempts = user.getFailedLoginAttempts() + 1;   // 6, 7, ... après expiration du verrou
if (attempts >= MAX_FAILED_ATTEMPTS) { /* reverrouille 15 min dès le 1er échec */ }
```

7.1.2 Analyse statique du backend (PMD / CPD)

Analyse statique backend : PMD 7.17 et CPD (copier-coller)

```
$ mvn pmd:pmd pmd:cpd
AuthController.java      L252  P3  EmptyCatchBlock               Avoid empty catch blocks
AuthController.java      L284  P3  EmptyCatchBlock               Avoid empty catch blocks
PollController.java      L5    P4  UnnecessaryImport             Unused import 'com.unigov.entity.User'
JwtUtils.java            L28   P4  UselessParentheses            Useless parentheses around `userPrincipal.getUsername()`.
JwtUtils.java            L30   P4  UselessParentheses            Useless parentheses around `new Date()`.
UserDetailsImpl.java     L119  P3  OverrideBothEqualsAndHashcode  Ensure you override both equals() and hashCode()
DecisionService.java     L4    P4  UnnecessaryImport             Unused import 'com.unigov.entity.Complaint'
DecisionService.java     L8    P4  UnnecessaryImport             Unused import 'com.unigov.entity.Poll'
DecisionService.java     L88   P3  EmptyCatchBlock               Avoid empty catch blocks
PollService.java         L10   P4  UnnecessaryImport             Unused import 'com.unigov.entity.Role'
--- CPD : 8 blocs dupliqués (17 lignes, 12 lignes, 12 lignes, 11 lignes, 11 lignes, 17 lignes, 10 lignes, 14 lignes)
```

Figure 9 – Résultat de PMD 7.17 et CPD sur le backend

Aucune violation de priorité 1 ou 2. Les points notables : trois blocs `catch` vides (des erreurs d'envoi d'e-mail sont silencieusement ignorées), `UserDetailsImpl` redéfinit `equals()` sans `hashCode()`, des imports inutilisés et 8 blocs dupliqués de 10 à 17 lignes.

7.1.3 Analyse statique du frontend (ESLint)

Analyse statique frontend : ESLint

```
$ npx eslint . --ext js,jsx

C:\Users\ASUS\Desktop\EniGov\frontend\src\App.jsx
  1:8  error  'React' is defined but never used  no-unused-vars

C:\Users\ASUS\Desktop\EniGov\frontend\src\components\DashboardLayout.jsx
  1:8  error  'React' is defined but never used  no-unused-vars

C:\Users\ASUS\Desktop\EniGov\frontend\src\components\Header.jsx
  1:8  error  'React' is defined but never used  no-unused-vars

C:\Users\ASUS\Desktop\EniGov\frontend\src\components\Logo.jsx
  1:8  error  'React' is defined but never used  no-unused-vars

C:\Users\ASUS\Desktop\EniGov\frontend\src\components\Sidebar.jsx
    1:8   error  'React' is defined but never used                              no-unused-vars
  158:42  error  `'` can be escaped with `&apos;`, `&lsquo;`, `&#39;`, `&rsquo;`  react/no-unescaped-entities

C:\Users\ASUS\Desktop\EniGov\frontend\src\context\AuthContext.jsx
   1:8   error    'React' is defined but never used                                                                                                  no-unused-vars
   1:54  error    'useEffect' is defined but never used                                                                                              no-unused-vars
  46:14  warning  Fast refresh only works when a file only exports components. Use a new file to share constants or functions between components  react-refresh/o
nly-export-components

   [...]

✖ 59 problems (55 errors, 4 warnings)
```

Figure 10 – Extrait du rapport ESLint (59 problèmes : 55 erreurs, 4 avertissements)

Répartition : 34 variables ou imports inutilisés, 19 caractères non échappés dans le JSX, 2 dépendances manquantes dans des `useEffect` (risque de données obsolètes à l'écran), 2 attributs inconnus. Ces défauts relèvent de la dette technique (**ANO-07**).

7.1.4 Analyse des dépendances (npm audit)

Audit des dépendances : npm audit

```
$ npm audit --workspace=frontend
# npm audit report

@babel/core  <=7.29.0
@babel/core: Arbitrary File Read via sourceMappingURL Comment - https://github.com/advisories/GHSA-4x5r-pxfx-6jf8
fix available via `npm audit fix`
node_modules/@babel/core

@babel/plugin-transform-modules-systemjs  7.12.0 - 7.29.0
Severity: high
@babel/plugin-transform-modules-systemjs generates arbitrary code when compiling malicious input - https://github.com/advisories/GHSA-fv7c-fp4j-7gwp
fix available via `npm audit fix`
node_modules/@babel/plugin-transform-modules-systemjs

@remix-run/router  1.3.0 - 1.23.2
   [...]
29 vulnerabilities (3 low, 11 moderate, 15 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force
```

Figure 11 – Audit des dépendances du frontend

29 vulnérabilités connues : 15 hautes, 11 modérées, 3 faibles. Elles touchent notamment `axios` (SSRF, contournement d'authentification) et `react-router` (redirection ouverte). L'exigence ENF-08 n'est pas satisfaite (**ANO-06**).

7.2 Tests unitaires (JUnit 5 + Mockito)

Les services sont testés isolément ; leurs dépendances (repositories, notifications) sont simulées avec Mockito. Exemple : la table de décision de la visibilité d'un sondage, implémentée par un test paramétré :

```
@ParameterizedTest
@CsvSource(nullValues = "-", value = {
    // targetAll, filiereCible, promoCible, filiereEtu, promoEtu, attendu
    "false, INFO, DEUXIEME_ANNEE, INFO, DEUXIEME_ANNEE, true",
    "false, INFO, DEUXIEME_ANNEE, INFO, PREMIERE_ANNEE, false",
    "false, INFO, DEUXIEME_ANNEE, GSIL, DEUXIEME_ANNEE, false",  /* ... 9 règles */ })
void visibilite(boolean targetAll, Filiere fCible, Promotion pCible, Filiere fEtu, Promotion pEtu,
boolean attendu) {
    Poll poll = new Poll();  poll.setTargetAll(targetAll);  /* ... */
    assertEquals(attendu, poll.isVisibleTo(fEtu, pEtu));
}
```

Cas	Classe de test	Tests	Technique	Itération 1	Itération 2
TU-01	PollServiceTest.java	11	Partitions d'équivalence, valeurs limites (index −1, 3, 99 ; 1 et 2 options)	Succès	—
TU-02	PollVisibilityTest.java	9	Table de décision (9 combinaisons filière × promotion)	Succès	—
TU-03	ComplaintServiceTest.java	5	Test de transitions d'états (aucun → pour → aucun, pour → contre)	Succès	—
TU-04	JwtUtilsTest.java	4	Partitions : jeton valide, signé avec une autre clé, expiré, mal formé	Échec	Succès

Le test `JwtUtilsTest.jetonSigneAvecAutreCle` a échoué lors de la première exécution : un jeton signé avec une autre clé provoque une `SignatureException` non interceptée au lieu d'un rejet propre (**ANO-02**). L'impact reste limité car le filtre HTTP intercepte l'exception (le test d'intégration TI-04 confirme une réponse 401/403), mais le contrat de la méthode n'est pas respecté.

7.3 Tests d'intégration (Spring Boot + MockMvc + PostgreSQL)

Le contexte Spring complet (sécurité, contrôleurs, services, JPA) est démarré sur une vraie base PostgreSQL de test. Les requêtes HTTP sont simulées avec MockMvc. Exemple : le test qui a révélé la fuite de confidentialité :

```
@Test @DisplayName("ANO-01 : un autre étudiant ne doit pas lire une réclamation privée par son
identifiant")
void priveeInaccessibleParId() throws Exception {
    Session a = nouvelEtudiant("a");   Session b = nouvelEtudiant("b");
    String id = creerReclamation(a, "Privée par id", false);
    mvc.perform(get("/api/complaints/" + id).header("Authorization",
b.bearer())).andExpect(status().isForbidden());
    mvc.perform(get("/api/complaints/" + id).header("Authorization",
a.bearer())).andExpect(status().isOk());
}
```

Cas	Objet	Itération 1	Itération 2	Commentaire
TI-01	Inscription : domaine et politique de mot de passe	Succès	—	6 valeurs limites + domaine + doublon conformes
TI-02	Vérification d'e-mail obligatoire avant connexion	Succès	—	403 puis 200 après vérification
TI-03	Verrouillage du compte après 5 échecs	Succès	—	4 × 401 puis 429 au 5e échec
TI-04	Matrice d'autorisations par rôle	Succès	—	6/6 contrôles d'accès conformes
TI-05	Confidentialité d'une réclamation privée (accès par identifiant)	Échec	Succès	Re-test après correctif ANO-01 : 403 pour un autre étudiant, 200 pour l'auteur et le délégué
TI-06	Traitement d'une réclamation et notification de l'étudiant	Succès	—	Statut RESOLVED + notification COMPLAINT_STATUS_CHANGED
TI-07	Vote unique à un sondage	Succès	—	Second vote refusé (400)
TI-08	Messagerie : étudiant vers étudiant interdit	Succès	—	Message étudiant → étudiant refusé (400)
TI-09	Remise à zéro du compteur après expiration du verrouillage	Échec	Bloqué	ANO-03 ouverte : test désactivé (@Disabled) en attendant le correctif

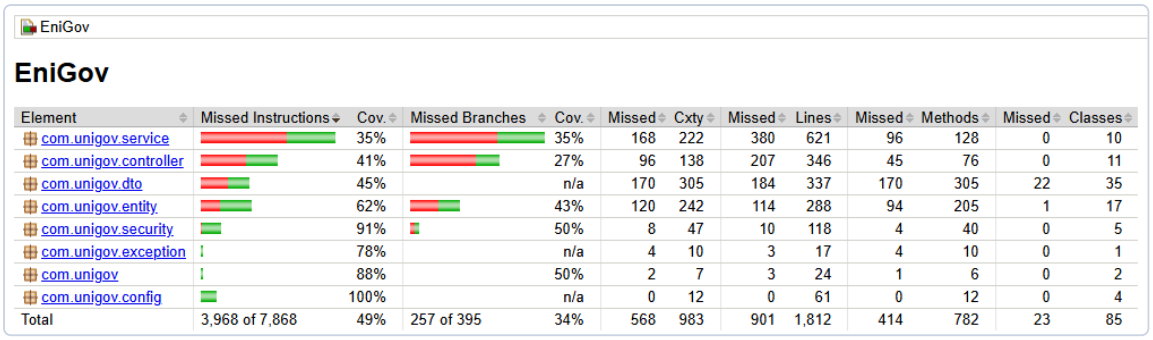
Exécution 1 : mvn test sur le build initial (v1.0.0)

```
$ cd backend && mvn test
--- jacoco:0.8.14:prepare-agent (default) @ unigov ---
[ERREUR] Tests run: 11, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 11.03 s <<< FAILURE! -- in com.unigov.integration.AuthIntegrationTest
[ERREUR] com.unigov.integration.AuthIntegrationTest.compteurRemisAZeroApresExpiration -- Time elapsed: 0.540 s <<< FAILURE!
org.opentest4j.AssertionFailedError: Le compte est reverrouillé dès le 1er échec : le compteur n'est jamais remis à zéro ==> expected: <401> but was: <429>
[ERREUR] Tests run: 5, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.894 s <<< FAILURE! -- in com.unigov.integration.ComplaintPollIntegrationTest
[ERREUR] com.unigov.integration.ComplaintPollIntegrationTest.priveeInaccessibleParId -- Time elapsed: 0.464 s <<< FAILURE!
java.lang.AssertionError: Status expected:<403> but was:<200>
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.589 s -- in com.unigov.integration.SecurityIntegrationTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.183 s -- in com.unigov.UniGovApplicationTests
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.303 s -- in com.unigov.unit.ComplaintServiceTest
[ERREUR] Tests run: 4, Failures: 0, Errors: 1, Skipped: 0, Time elapsed: 0.023 s <<< FAILURE! -- in com.unigov.unit.JwtUtilsTest
[ERREUR] com.unigov.unit.JwtUtilsTest.jetonSigneAvecAutreCle -- Time elapsed: 0.004 s <<< ERROR!
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.093 s -- in com.unigov.unit.PollServiceTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in com.unigov.unit.PollVisibilityTest
[ERREUR] Failures:
[ERREUR]   AuthIntegrationTest.compteurRemisAZeroApresExpiration:96 Le compte est reverrouillé dès le 1er échec : le compteur n'est jamais remis à zéro ==> expec
ted: <401> but was: <429>
[ERREUR]   ComplaintPollIntegrationTest.priveeInaccessibleParId:51 Status expected:<403> but was:<200>
[ERREUR] Errors:
[ERREUR]   JwtUtilsTest.jetonSigneAvecAutreCle:55 » Signature JWT signature does not match locally computed signature. JWT validity cannot be asserted and should
not be trusted.
[ERREUR] Tests run: 52, Failures: 2, Errors: 1, Skipped: 0
BUILD FAILURE
```

Figure 12 – Itération 1 : 52 tests, 3 en échec (ANO-01, ANO-02, ANO-03)

7.4 Tests structurels (couverture de code avec JaCoCo)

Le plugin JaCoCo instrumente le bytecode pendant `mvn test` et mesure les instructions et les branches exécutées par les tests unitaires et d'intégration.

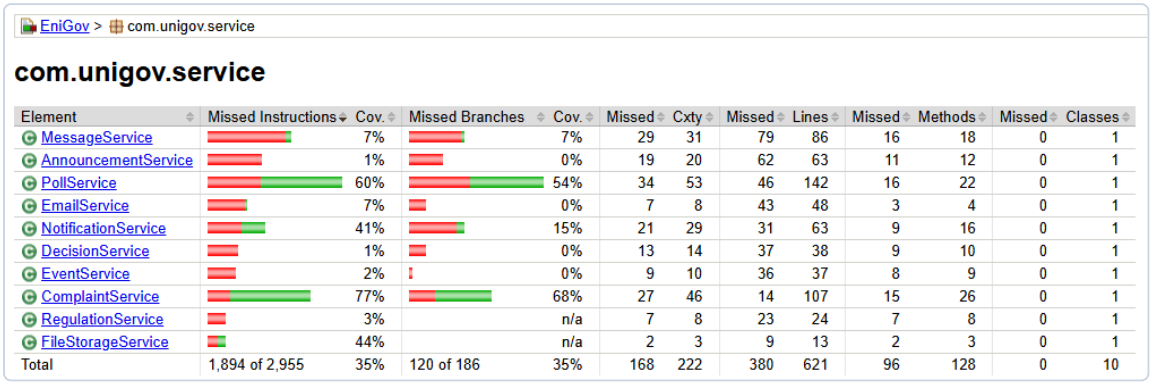


EniGov

EniGov

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.unigov.service		35%		35%	168	222	380	621	96	128	0	10
com.unigov.controller		41%		27%	96	138	207	346	45	76	0	11
com.unigov.dto		45%		n/a	170	305	184	337	170	305	22	35
com.unigov.entity		62%		43%	120	242	114	288	94	205	1	17
com.unigov.security		91%		50%	8	47	10	118	4	40	0	5
com.unigov.exception		78%		n/a	4	10	3	17	4	10	0	1
com.unigov		88%		50%	2	7	3	24	1	6	0	2
com.unigov.config		100%		n/a	0	12	0	61	0	12	0	4
Total	3,968 of 7,868	49%	257 of 395	34%	568	983	901	1,812	414	782	23	85

Figure 13 – Couverture globale du backend : 49 % des instructions, 34 % des branches



EniGov > com.unigov.service

com.unigov.service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
MessageService		7%		7%	29	31	79	86	16	18	0	1
AnnouncementService		1%		0%	19	20	62	63	11	12	0	1
PollService		60%		54%	34	53	46	142	16	22	0	1
EmailService		7%		0%	7	8	43	48	3	4	0	1
NotificationService		41%		15%	21	29	31	63	9	16	0	1
DecisionService		1%		0%	13	14	37	38	9	10	0	1
EventService		2%		0%	9	10	36	37	8	9	0	1
ComplaintService		77%		68%	27	46	14	107	15	26	0	1
RegulationService		3%		n/a	7	8	23	24	7	8	0	1
FileStorageService		44%		n/a	2	3	9	13	2	3	0	1
Total	1,894 of 2,955	35%	120 of 186	35%	168	222	380	621	96	128	0	10

Figure 14 – Couverture par service : les services ciblés (PollService, ComplaintService) dépassent 50 %

Analyse : l'objectif de 40 % des instructions est atteint (49 %). La couverture est concentrée sur les modules à risque P1 (réclamations 77 %, sondages 60 %, sécurité 91 %). Les services de messagerie, d'annonces et d'événements sont surtout exercés par les tests E2E, que JaCoCo ne mesure pas dans cette configuration. La vue détaillée montre aussi des branches non exécutées : `getAllComplaints` n'est appelée par aucun test JUnit, ce qui indique les prochains tests à écrire.



```
38.         complaint.setTitle(request.getTitle());
39.         complaint.setDescription(request.getDescription());
40.         complaint.setPublic(request.isPublic());
41.         complaint.setStatus(ComplaintStatus.PENDING);
42.         complaint.setStudentId(student.getId());
43.         complaint.setStudentName(student.getFullName());
44.         complaint.setAttachmentPath(attachmentUrl); // URL Cloudinary
45.
46.         Complaint saved = complaintRepository.save(complaint);
47.         return mapToResponse(saved, student.getId());
48.     }
49.
50.     public List<ComplaintResponse> getMyComplaints(String username) {
51.         User student = userRepository.findByUsername(username)
52.                 .orElseThrow(() -> new RuntimeException("Utilisateur non trouvé"));
53.         return complaintRepository.findByStudentIdOrderByCreatedAtDesc(student.getId()).stream()
54.                 .map(c -> mapToResponse(c, student.getId()))
55.                 .collect(Collectors.toList());
56.     }
57.
58.     public List<ComplaintResponse> getPublicComplaints(String username) {
59.         User user = userRepository.findByUsername(username)
60.                 .orElseThrow(() -> new RuntimeException("Utilisateur non trouvé"));
61.         return complaintRepository.findByIsPublicTrueOrderByCreatedAtDesc().stream()
62.                 .map(c -> mapToResponse(c, user.getId()))
63.                 .collect(Collectors.toList());
64.     }
65.
66.     public List<ComplaintResponse> getAllComplaints(String username) {
67.         User user = userRepository.findByUsername(username)
68.                 .orElseThrow(() -> new RuntimeException("Utilisateur non trouvé"));
69.         return complaintRepository.findAllByOrderByCreatedAtDesc().stream()
70.                 .map(c -> mapToResponse(c, user.getId()))
71.                 .collect(Collectors.toList());
72.     }
73.
74.     public ComplaintResponse getById(String id, String username) {
75.         User user = userRepository.findByUsername(username)
76.                 .orElseThrow(() -> new RuntimeException("Utilisateur non trouvé"));
77.         Complaint complaint = complaintRepository.findById(id)
78.                 .orElseThrow(() -> new RuntimeException("Réclamation non trouvée"));
79.         boolean autorise = complaint.isPublic()
80.                 || user.getId().equals(complaint.getStudentId())
81.                 || user.getRole() == Role.ROLE_DELEGUE;
82.         if (!autorise) {
83.             throw new AccessDeniedException("Réclamation privée");
84.         }
85.         return mapToResponse(complaint, user.getId());
86.     }
87.
88.     @Transactional
89.     public ComplaintResponse updateStatus(String id, ComplaintUpdate update) {
90.         Complaint complaint = complaintRepository.findById(id)
91.                 .orElseThrow(() -> new RuntimeException("Réclamation non trouvée"));
```

Figure 15 – Vue source JaCoCo : lignes couvertes (vert), non couvertes (rouge), branches partielles (jaune)

7.5 Tests système (E2E avec Playwright)

Les scénarios sont joués dans un vrai navigateur (Chromium) sur l'application complète : frontend, backend et base de données. Ils enchaînent plusieurs acteurs (le délégué publie, l'étudiant consulte).

Tests système E2E : Playwright (Chromium desktop + mobile Pixel 7)

```
$ npx playwright test -c e2e/playwright.config.js
Running 7 tests using 1 worker
  ok 1 [chromium-desktop] › e2e\specs\enigov.spec.js:23:1 › TS-01 Connexion du délégué et affichage du tableau de bord (1.5s)
  ok 2 [chromium-desktop] › e2e\specs\enigov.spec.js:31:1 › TS-02 Connexion refusée avec un mauvais mot de passe (1.4s)
  ok 3 [chromium-desktop] › e2e\specs\enigov.spec.js:38:1 › TS-03 Le délégué publie une annonce, visible par un étudiant (3.1s)
  ok 4 [chromium-desktop] › e2e\specs\enigov.spec.js:58:1 › TS-04 Parcours réclamation : création par l'étudiant, traitement par le délégué (4.6s)
  ok 5 [chromium-desktop] › e2e\specs\enigov.spec.js:90:1 › TS-05 Sondage : création par le délégué et vote d'un étudiant (2.9s)
  ok 6 [chromium-desktop] › e2e\specs\enigov.spec.js:110:1 › TS-06 Contrôle d'accès : un étudiant ne peut pas ouvrir la page Étudiants (1.5s)
  x  7 [chromium-mobile] › e2e\specs\enigov.spec.js:119:1 › TS-07 @mobile Affichage responsive sur smartphone (Pixel 7) (3.4s)
    Error: aucun défilement horizontal (largeur en trop, px)
    Expected: <= 0
    Received:    107
  1 failed
  6 passed (19.9s)
```

Figure 16 – Exécution de la suite E2E : 6 réussis, 1 échec (TS-07)

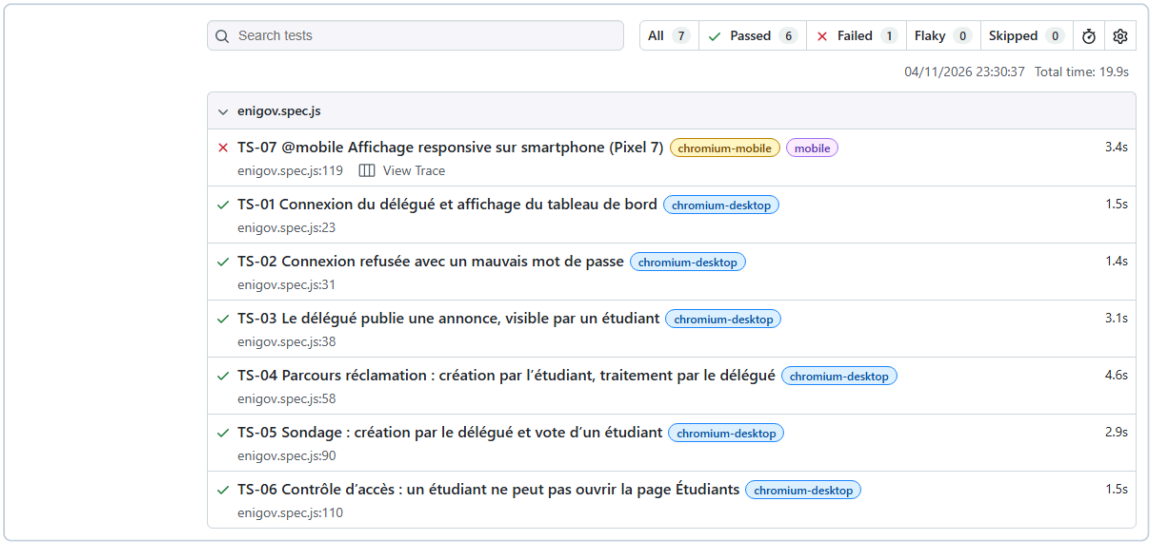


Figure 17 – Rapport HTML Playwright

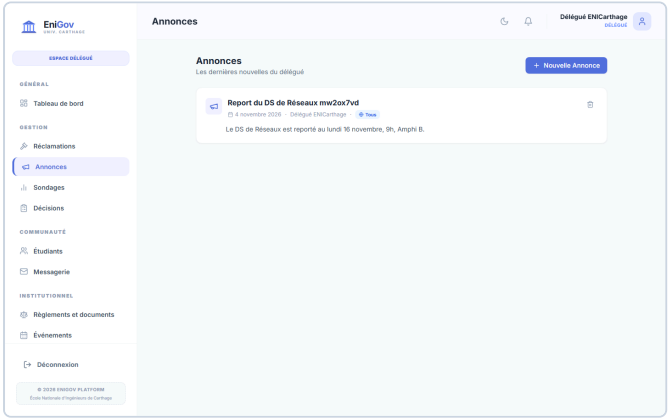


Figure 18 – TS-03 : le délégué publie une annonce

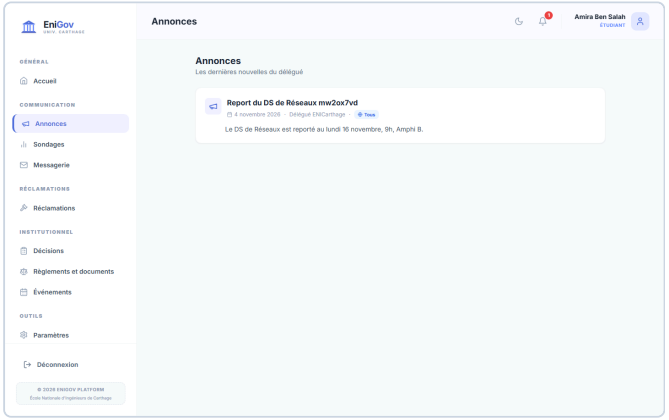


Figure 19 – TS-03 : l'étudiante la voit, sans bouton de création

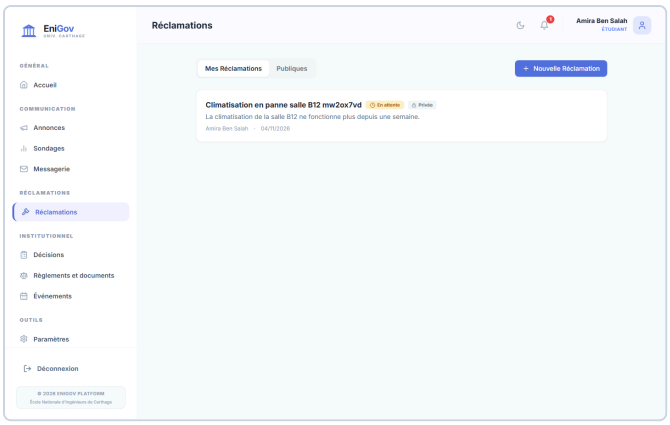


Figure 20 – TS-04 : réclamation créée, statut « En attente »

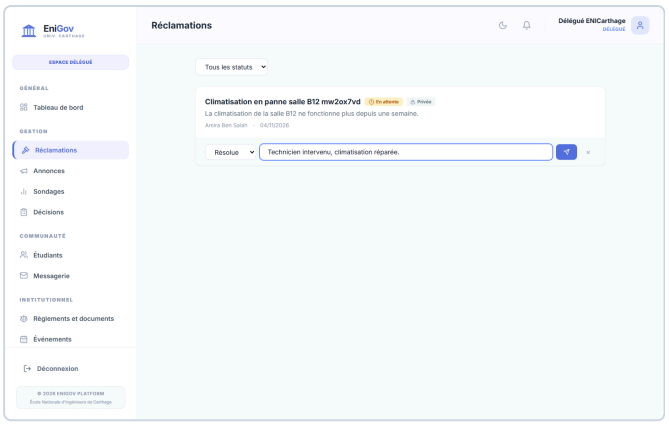


Figure 21 – TS-04 : le délégué passe le statut à « Résolue » et répond

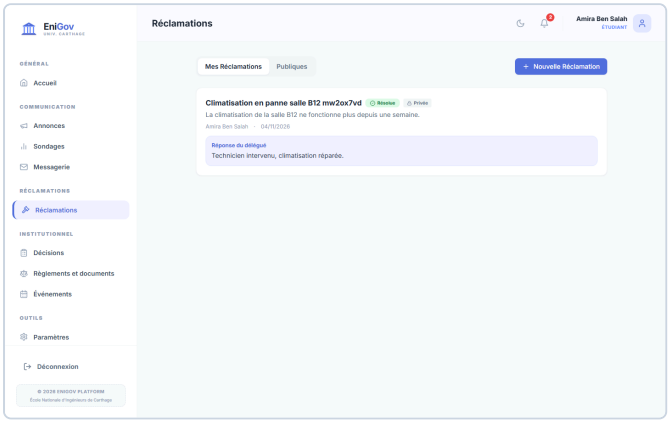


Figure 22 – TS-04 : l'étudiante voit le statut « Résolue » et la réponse

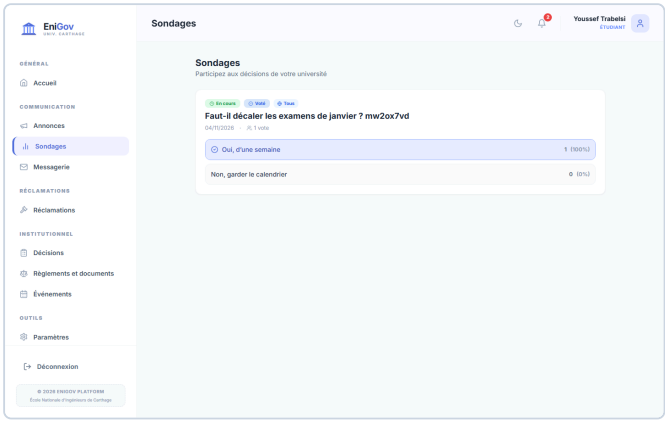


Figure 23 – TS-05 : vote d'un étudiant, résultat 100 %

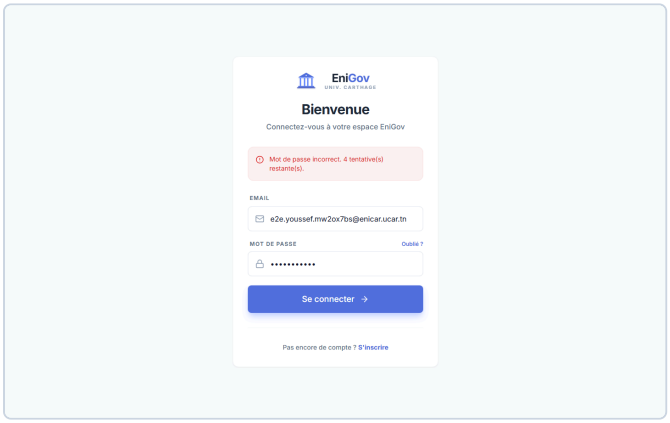


Figure 24 – TS-02 : mauvais mot de passe, 4 tentatives restantes

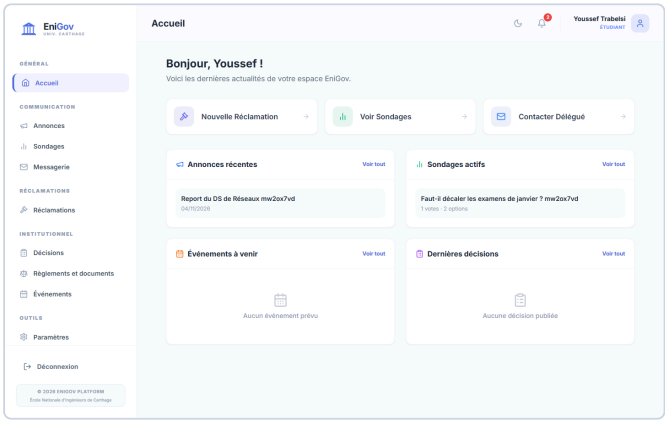


Figure 25 – TS-06 : un étudiant qui ouvre /students est redirigé vers l'accueil

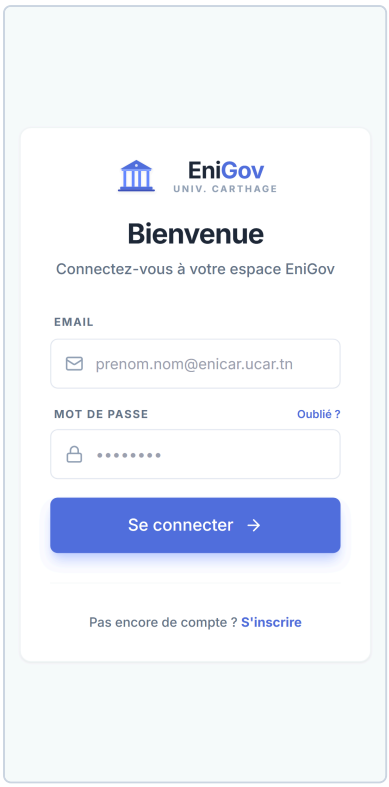


Figure 26 – TS-07 : connexion sur Pixel 7

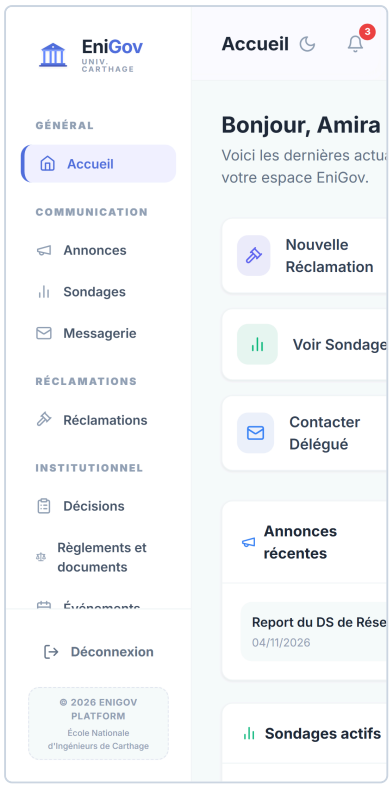


Figure 27 – TS-07 : accueil sur Pixel 7

TS-07 en échec – ANO-05. Sur smartphone, la barre latérale reste affichée en permanence et occupe près de la moitié de l'écran ; le contenu déborde de **107 px** horizontalement (titres et cartes coupés). ENF-05 n'est pas satisfaite.

7.6 Tests d'acceptation manuels

Les fiches de test TA-01 à TA-06 ont été rédigées dans Squash TM (pas de test : action → résultat attendu), puis exécutées pas à pas dans le navigateur du point de vue de l'utilisateur. Chaque étape a été capturée comme preuve ; le script `qa/acceptance/run-acceptance.mjs` a servi à enregistrer ces preuves. Les résultats ont ensuite été saisis dans l'exécution Squash TM correspondante.

TA-01 — Inscription d'un nouvel étudiant (EF-01, EF-02)

#	Action	Résultat attendu	Résultat obtenu
1	Ouvrir /register et s'inscrire avec une adresse @gmail.com	Inscription refusée : seuls les e-mails @enicar.ucar.tn sont acceptés	**Succès** Conforme au résultat attendu
2	Saisir une adresse valide avec un mot de passe faible « motdepasse »	Indicateur « Faible — min. 8 caractères, 1 majuscule, 1 chiffre » et bouton S'inscrire désactivé	**Succès** Conforme au résultat attendu
3	Saisir le mot de passe « Etudiant2026 » et valider	Message « Vérifiez votre email ! » affiché	**Succès** Conforme au résultat attendu
4	Tenter de se connecter avant d'avoir cliqué sur le lien de vérification	Connexion refusée : e-mail non vérifié	**Succès** Conforme au résultat attendu

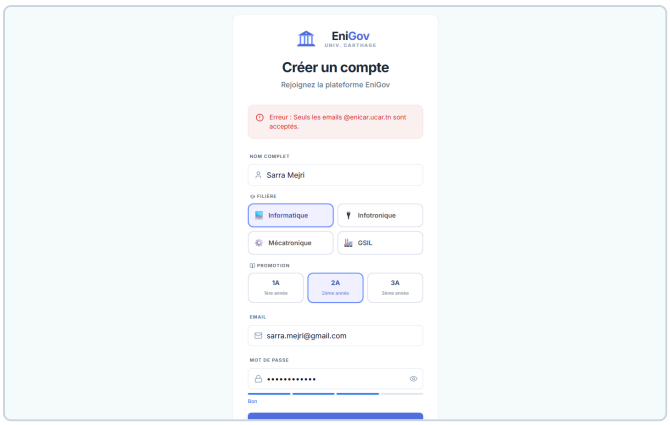


Figure 28 – Étape 1 : adresse @gmail.com refusée

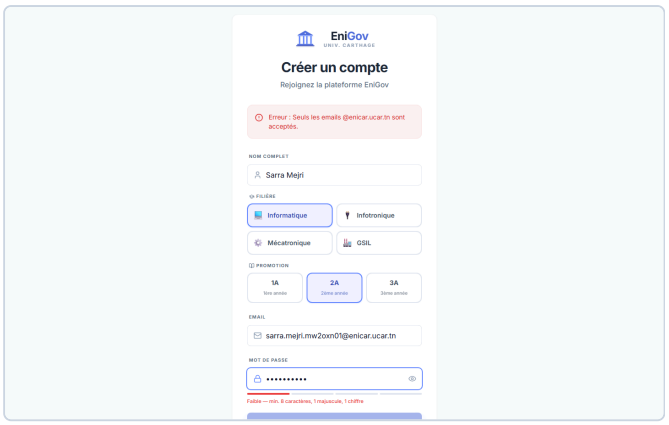


Figure 29 – Étape 2 : mot de passe faible, bouton désactivé

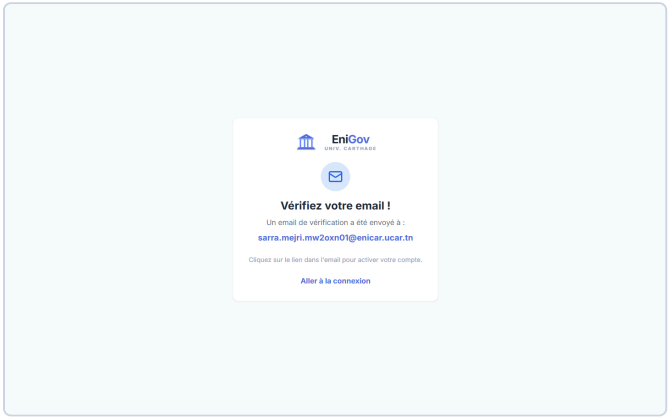


Figure 30 – Étape 3 : inscription acceptée, vérification demandée

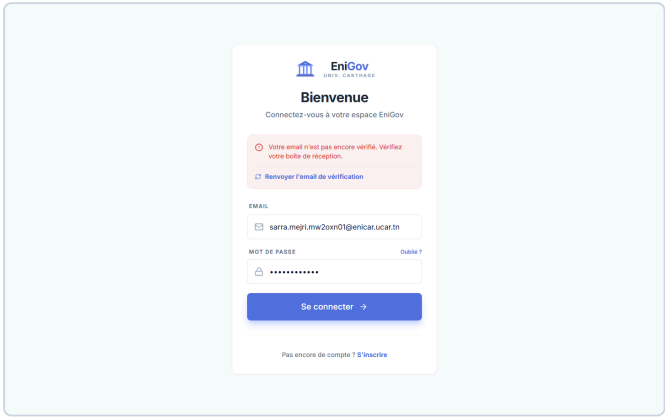


Figure 31 – Étape 4 : connexion refusée tant que l'e-mail n'est pas vérifié

TA-02 — Réclamation publique et vote (EF-05, EF-07)

#	Action	Résultat attendu	Résultat obtenu
1	Amira crée une réclamation PUBLIQUE « Wi-Fi instable au bloc B »	Réclamation créée, statut « En attente », badge « Publique »	**Succès** Conforme au résultat attendu
2	Youssef ouvre l'onglet « Publiques » et vote pour (pouce levé)	La réclamation est visible avec le nom de l'auteur ; score = 1	**Succès** Conforme au résultat attendu
3	Youssef clique à nouveau sur le pouce levé	Le vote est annulé (bascule) ; score = 0	**Succès** Conforme au résultat attendu

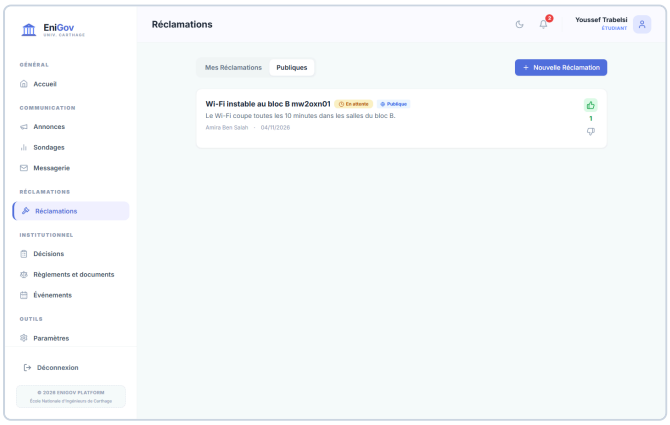


Figure 32 – Vote « pour » : score = 1 (pouce vert)

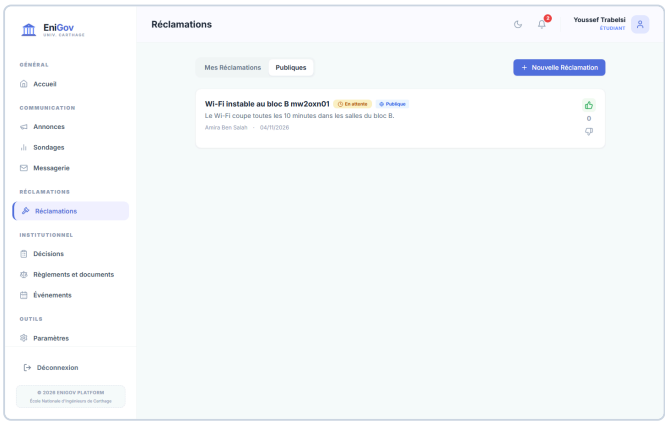


Figure 33 – Second clic : vote annulé, score = 0

TA-03 — Message au délégué et notification (EF-13, EF-16)

#	Action	Résultat attendu	Résultat obtenu
1	Youssef ouvre Messagerie et clique sur « Nouveau message »	La liste de contacts ne propose que le délégué (pas les autres étudiants)	**Succès** Conforme au résultat attendu
2	Sélectionner le délégué, écrire un message et l'envoyer	Le message apparaît dans la conversation	**Succès** Conforme au résultat attendu
3	Le délégué se connecte et ouvre la conversation	Badge de notification sur la cloche ; message reçu de Youssef	**Succès** Conforme au résultat attendu

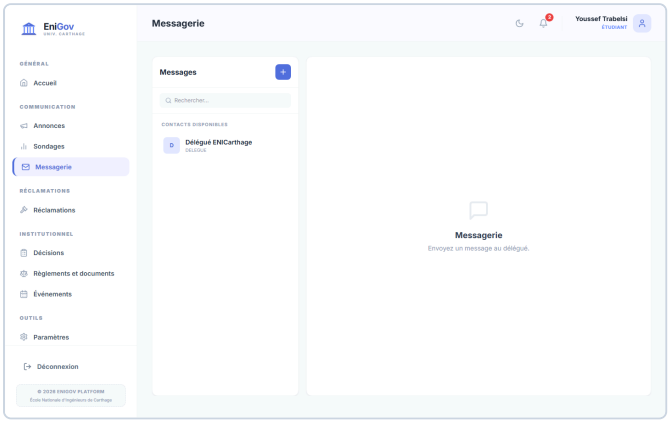


Figure 34 – L'étudiant ne peut contacter que le délégué

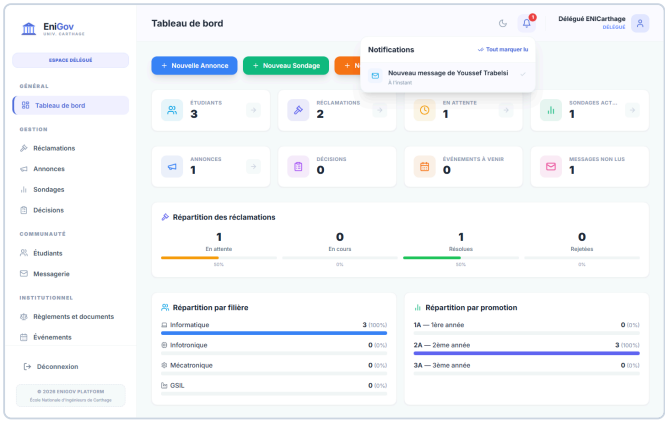


Figure 35 – Le délégué reçoit une notification « Nouveau message »

TA-04 — Calendrier des événements (EF-14)

#	Action	Résultat attendu	Résultat obtenu
1	Le délégué crée l'événement « Forum des clubs » (15/12/2026 10:00, Hall principal)	L'événement apparaît dans la liste	**Succès** Conforme au résultat attendu
2	Amira consulte la page Événements	L'événement est visible ; aucun bouton de création pour l'étudiant	**Succès** Conforme au résultat attendu

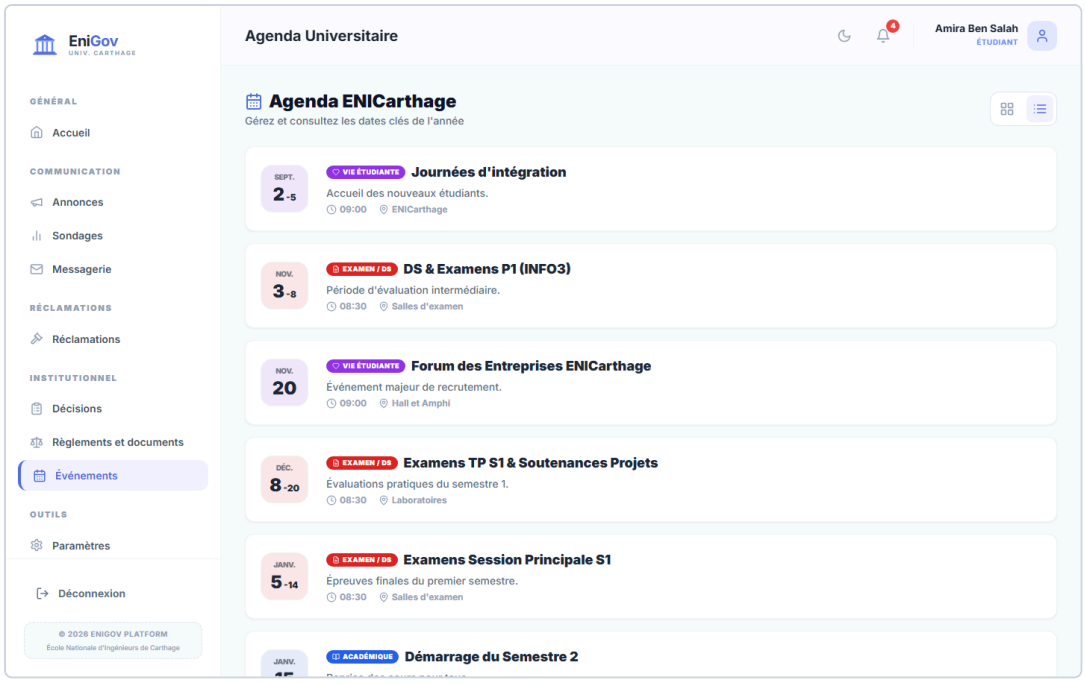


Figure 36 – L'étudiante voit l'événement créé par le délégué

TA-05 — Thème clair / sombre (ENF-05)

#	Action	Résultat attendu	Résultat obtenu
1	Cliquer sur l'icône lune dans l'en-tête	L'interface passe en thème sombre	Succès Conforme au résultat attendu
2	Recharger la page (F5)	Le thème sombre est conservé	Succès Conforme au résultat attendu

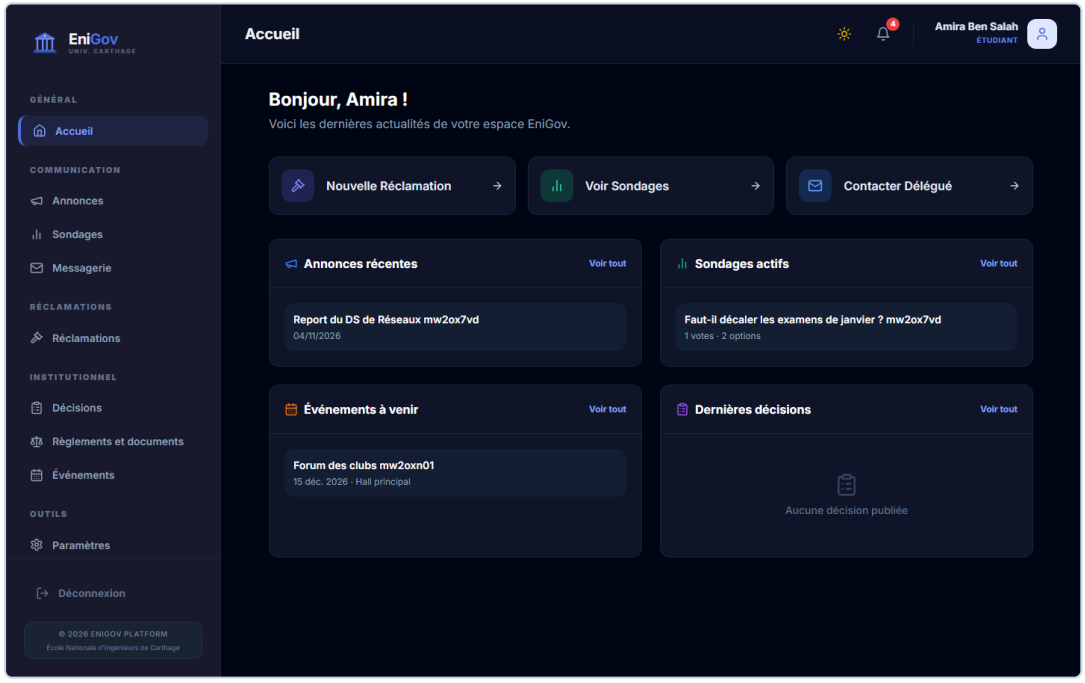


Figure 37 – Thème sombre conservé après rechargement

TA-06 — Persistance des événements après redémarrage (ENF-09)

Test exploratoire guidé par la revue de code (§ 7.1.1) : on crée un événement, on redémarre le backend et on vérifie qu'il existe toujours.

TA-06 : vérification manuelle de la persistance des événements (API + redémarrage)

```
$ # TA-07 Etape 1 : le delegue cree un evenement "Reunion TEST-ANO04"
$ curl -X POST http://localhost:8081/api/events -H "Authorization: Bearer <jeton delegue>" -d {"title":"Reunion TEST-ANO04","date":"2026-12-10T10:00:00","locatio
n":"Amphi A",...}
{"id":"6fa1f6a8-d89b-4b86-bdc4-fdf843b6bc88","title":"Reunion TEST-ANO04","date":"2026-12-10T10:00:00","location":"Amphi A","type":"ACADEMIC"}  -> HTTP 200

$ # Etape 2 : verification avant redemarrage
$ curl -s http://localhost:8081/api/events | grep -c "TEST-ANO04"   =>  1
$ nombre total d evenements  =>  12

$ # Etape 3 : redemarrage du backend (mvn spring-boot:run)
INFO 30008 --- [enigov] [  restartedMain] com.unigov.UniGovApplication            : Started UniGovApplication in 5.441 seconds (process run
>>> SEEDER : Calendrier mis a jour avec les periodes (Debut/Fin).  [eventRepository.deleteAll() + seedEvents()]

$ # Etape 4 : verification apres redemarrage
$ curl -s http://localhost:8081/api/events | grep -c "TEST-ANO04"   =>  0
$ nombre total d evenements  =>  11

RESULTAT : ECHEC - 1 evenement cree par le delegue a ete supprime au redemarrage (ANO-04).
Cause (revue de code) : DatabaseSeeder.run() appelle eventRepository.deleteAll() a chaque demarrage.
```

Figure 38 – TA-06 en échec : l'événement créé disparaît au redémarrage (ANO-04)

7.7 Tests non fonctionnels

Performance et fiabilité : test de charge k6 (TNF-01)

Profil de charge : montée à 20 utilisateurs virtuels en 10 s, palier de 40 s, descente en 10 s. Chaque utilisateur consulte cinq ressources (annonces, événements, sondages, réclamations, statistiques), puis marque une pause d'une seconde. Seuils : p(95) < 500 ms (ENF-03) et taux d'échec < 1 % (ENF-04).

Test de charge k6 : 20 utilisateurs virtuels pendant 60 s

```
$ docker run --rm -v ./qa/perf:/scripts grafana/k6 run /scripts/charge-api.js
     execution: local
        script: /scripts/charge-api.js
        output: -
     scenarios: (100.00%) 1 scenario, 20 max VUs, 1m30s max duration (incl. graceful stop):
              * default: Up to 20 looping VUs for 1m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)
  ▌ THRESHOLDS
    http_req_duration
    ✓ 'p(95)<500' p(95)=35.74ms
    http_req_failed
    ✓ 'rate<0.01' rate=0.02%
  ▌ TOTAL RESULTS
    checks_total.......: 4631   56.225035/s
    checks_succeeded...: 99.97% 4630 out of 4631
    checks_failed......: 0.02%  1 out of 4631
    ✓ connexion OK
    ✗ GET /announcements = 200
      ↳  99% — ✓ 925 / ✗ 1
    ✓ GET /events = 200
    ✓ GET /polls = 200
    ✓ GET /complaints = 200
    ✓ GET /stats = 200
    HTTP
    http_req_duration..............: avg=18.75ms min=0s     med=15.24ms max=247.09ms p(90)=30.91ms p(95)=35.74ms
      { expected_response:true }...: avg=18.76ms min=4.42ms med=15.24ms max=247.09ms p(90)=30.91ms p(95)=35.74ms
    http_req_failed................: 0.02%  1 out of 4631
    http_reqs......................: 4631   56.225035/s
    EXECUTION
    iteration_duration.............: avg=1.12s   min=1.04s  med=1.08s   max=31.07s   p(90)=1.1s    p(95)=1.14s
    iterations.....................: 926    11.242579/s
    vus............................: 1      min=1        max=20
    vus_max........................: 20     min=20       max=20
    NETWORK
    data_received..................: 6.3 MB 77 kB/s
    data_sent......................: 1.2 MB 14 kB/s
```

Figure 39 – Résultat k6 : les deux seuils sont respectés

Résultat : 4 631 requêtes (56 req/s) ; temps de réponse moyen 18,8 ms, p(95) = 35,7 ms, maximum 247 ms. Une seule requête a échoué (0,02 %), sur un délai réseau entre le conteneur k6 et l'hôte (*i/o timeout*), et non sur une erreur applicative. ENF-03 et ENF-04 sont satisfaites avec une large marge.

Sécurité

- Matrice d'autorisations (TI-04, TS-06) : sans jeton, avec un jeton falsifié ou avec le rôle étudiant, les ressources du délégué sont refusées (401/403, jamais 500). **Conforme**
- Verrouillage après 5 échecs (TI-03) **Conforme** ; reverrouillage abusif après expiration (TI-09) **ANO-03**
- Confidentialité des réclamations privées (TI-05) **ANO-01**, corrigée puis re-testée **Conforme**
- Dépendances vulnérables (TNF-02) **ANO-06**

Ergonomie, compatibilité et maintenabilité

- Responsive (TS-07) : **ANO-05** ; thème clair/sombre mémorisé (TA-05) **Conforme**
- Maintenabilité (TNF-03, TNF-04) : couverture 49 % **Conforme** ; analyse statique **ANO-07**

7.8 Tests liés aux changements et tests de maintenance

Correctifs appliqués

Les deux anomalies corrigeables sans changement de conception ont été corrigées : **ANO-01** (contrôle du propriétaire dans `ComplaintService.getById`) et **ANO-02** (interception de `SecurityException` dans `JwtUtils`).

Correctifs ANO-01 et ANO-02 (git diff)

```
$ git diff backend/src/main
diff --git a/backend/src/main/java/com/unigov/security/JwtUtils.java b/backend/src/main/java/com/unigov/security/JwtUtils.java
index 668890c..07e5768 100644
--- a/backend/src/main/java/com/unigov/security/JwtUtils.java
+++ b/backend/src/main/java/com/unigov/security/JwtUtils.java
@@ -45,6 +45,8 @@ public class JwtUtils {
         try {
             Jwts.parserBuilder().setSigningKey(key()).build().parse(authToken);
             return true;
+        } catch (io.jsonwebtoken.security.SecurityException e) {
+            logger.error("Invalid JWT signature: {}", e.getMessage());
         } catch (MalformedJwtException e) {
             logger.error("Invalid JWT token: {}", e.getMessage());
         } catch (ExpiredJwtException e) {
diff --git a/backend/src/main/java/com/unigov/service/ComplaintService.java b/backend/src/main/java/com/unigov/service/ComplaintService.java
index 16ef5fe..371e9e3 100644
--- a/backend/src/main/java/com/unigov/service/ComplaintService.java
+++ b/backend/src/main/java/com/unigov/service/ComplaintService.java
@@ -4,10 +4,12 @@ import com.unigov.dto.ComplaintDtos.*;
 import com.unigov.entity.Complaint;
 import com.unigov.entity.ComplaintEnums.ComplaintStatus;
 import com.unigov.entity.Notification.NotificationType;
+import com.unigov.entity.Role;
 import com.unigov.entity.User;
 import com.unigov.repository.ComplaintRepository;
 import com.unigov.repository.UserRepository;
 import org.springframework.beans.factory.annotation.Autowired;
+import org.springframework.security.access.AccessDeniedException;
 import org.springframework.stereotype.Service;
 import org.springframework.transaction.annotation.Transactional;

@@ -74,6 +76,12 @@ public class ComplaintService {
                 .orElseThrow(() -> new RuntimeException("Utilisateur non trouvé"));
         Complaint complaint = complaintRepository.findById(id)
                 .orElseThrow(() -> new RuntimeException("Réclamation non trouvée"));
+        boolean autorise = complaint.isPublic()
+                || user.getId().equals(complaint.getStudentId())
+                || user.getRole() == Role.ROLE_DELEGUE;
+        if (!autorise) {
+            throw new AccessDeniedException("Réclamation privée");
+        }
         return mapToResponse(complaint, user.getId());
     }
```

Figure 40 – Modifications apportées au code (git diff)

Re-test de confirmation (TR-01) et non-régression (TR-02)

Le **re-test** ré-exécute les cas qui avaient échoué (TU-04, TI-05) pour confirmer la correction. La **non-régression** ré-exécute toute la suite automatisée (52 tests JUnit et 7 tests E2E) pour vérifier que les modifications n'ont rien cassé. TI-09 (ANO-03, non corrigée) est désactivé avec `@Disabled` et une justification, puis marqué « Bloqué » dans Squash TM.

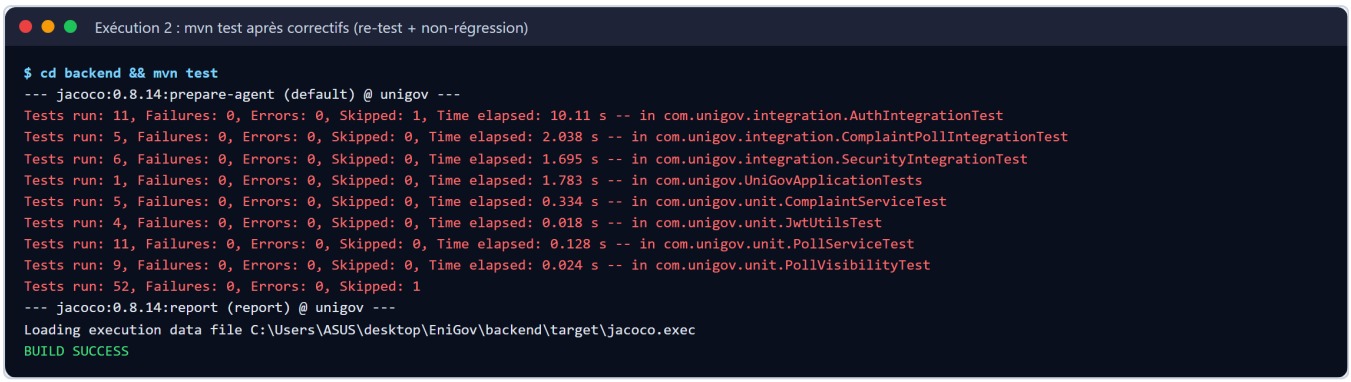


Figure 41 – Itération 2 : 52 tests, 51 réussis, 1 ignoré (ANO-03), BUILD SUCCESS

La vue source JaCoCo (figure 15) montre que le nouveau code du correctif ANO-01 est entièrement exécuté par les tests (branches en vert).

Tests de maintenance (TM-01)

Le projet cible Java 17 et une base PostgreSQL locale. Il a été porté vers un nouvel environnement : **JDK 25** et **PostgreSQL 16 sous Docker**. La compilation et l'ensemble des tests passent. Un seul ajustement a été nécessaire : Mockito/Byte Buddy 1.14 ne reconnaît pas officiellement le JDK 25, d'où l'option `-Dnet.bytebuddy.experimental=true` ajoutée au `pom.xml`. Une mise à jour de Spring Boot (≥ 3.5) supprimerait ce contournement.

7.9 Suivi de la campagne dans Squash TM

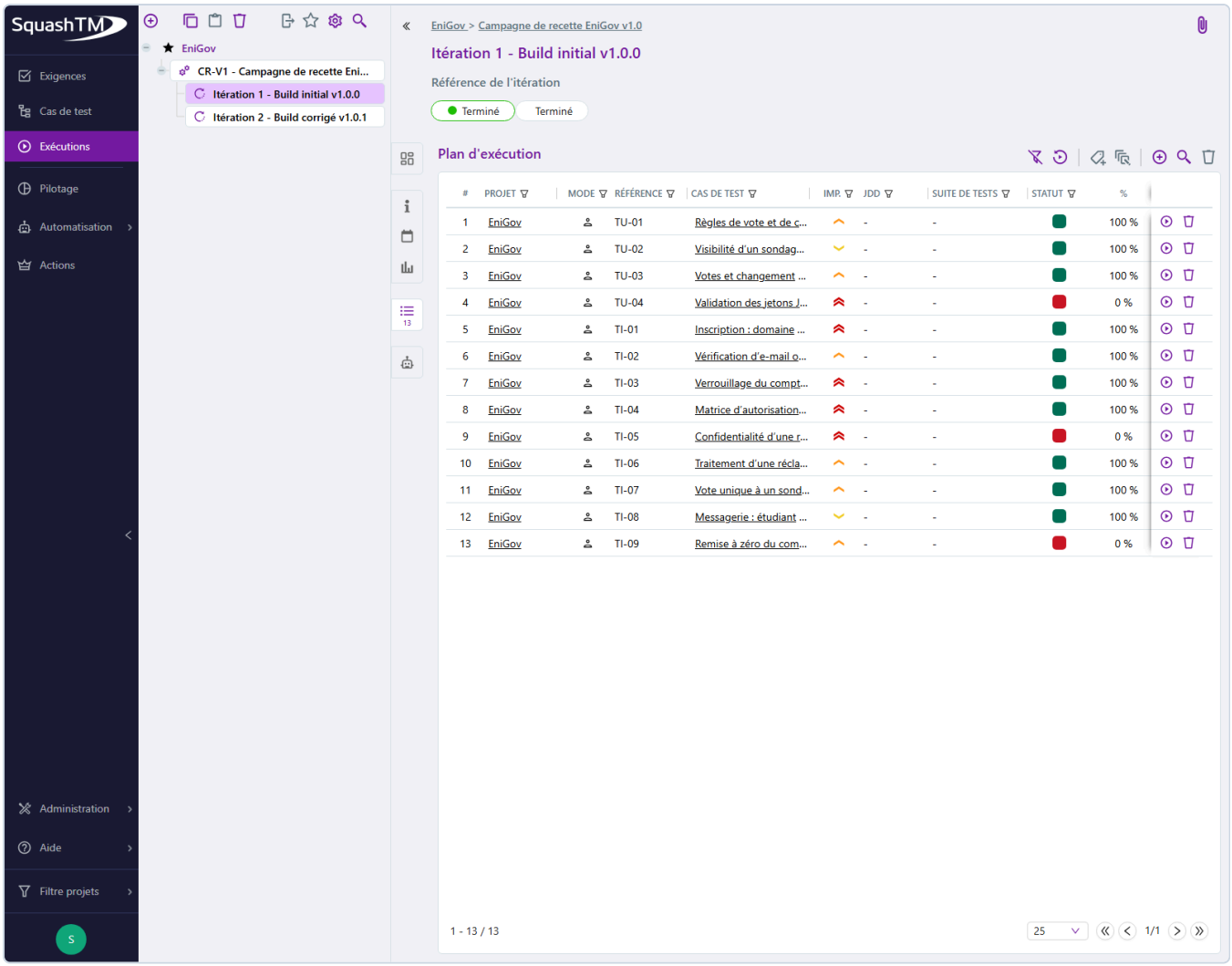


Figure 42 – Itération 1 (build v1.0.0) : plan d'exécution et statuts

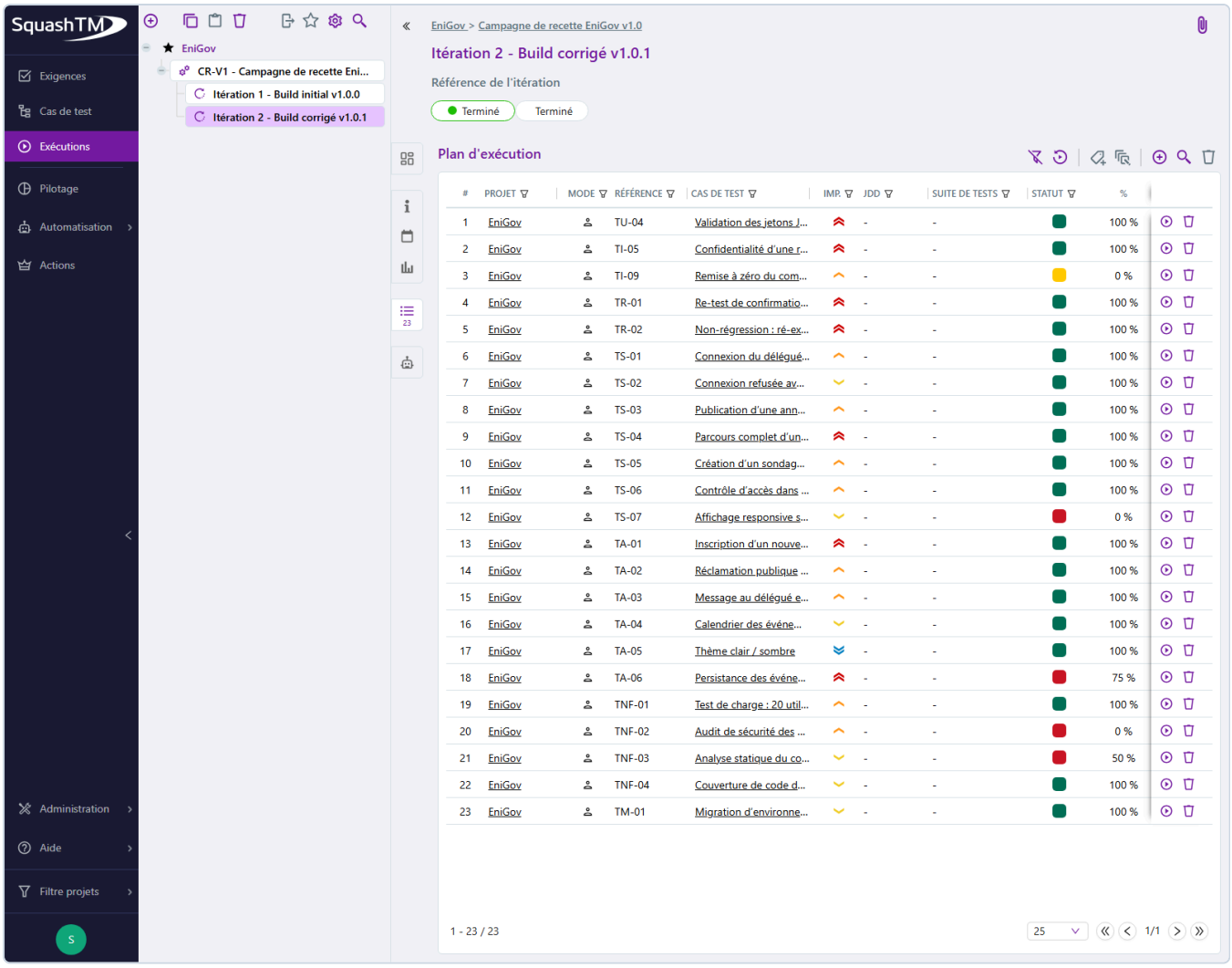


Figure 43 – Itération 2 (build v1.0.1) : plan d'exécution et statuts

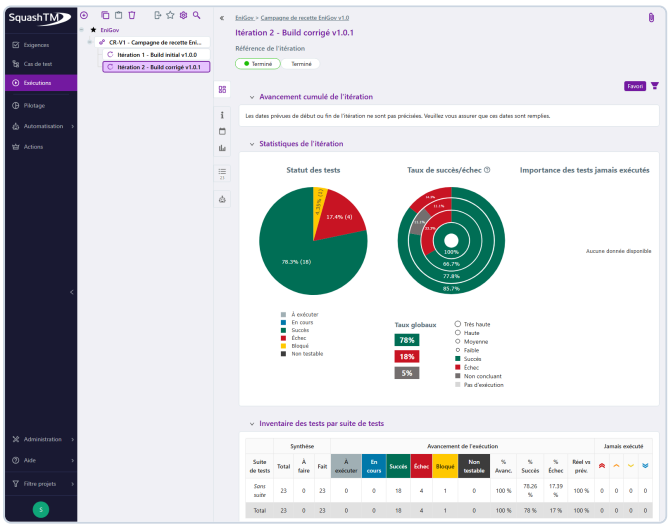


Figure 44 – Tableau de bord de l'itération 2

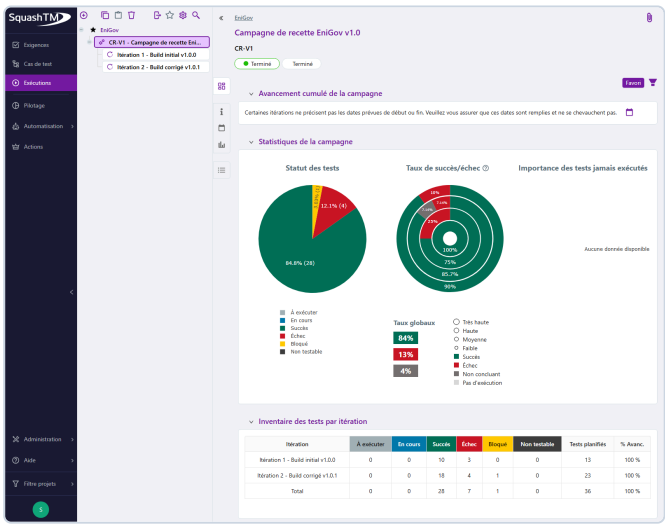


Figure 45 – Tableau de bord de la campagne

8. Bilan : anomalies, métriques et verdict de recette

8.1 Anomalies détectées

Réf.	Gravité	Description	Détectée par	Exigence	Statut
ANO-01	**Critique**	Fuite de confidentialité : une réclamation privée est lisible par n'importe quel étudiant via GET /api/complaints/{id} (IDOR)	Revue de code + TI-05	EF-06	**Corrigée (v1.0.1)**
ANO-02	**Mineure**	JwtUtils.validateJwtToken lève SignatureException au lieu de renvoyer false pour un jeton signé avec une autre clé	TU-04	ENF-01	**Corrigée (v1.0.1)**
ANO-03	**Majeure**	Le compteur d'échecs n'est jamais remis à zéro après expiration du verrouillage : un seul échec reverrouille le compte 15 min	Revue de code + TI-09	EF-03	Ouverte
ANO-04	**Majeure**	DatabaseSeeder supprime tous les événements à chaque démarrage (eventRepository.deleteAll()) : perte des événements créés par le délégué	Revue de code + TA-06	ENF-09	Ouverte
ANO-05	**Majeure**	Mobile : la barre latérale n'est pas repliable, débordement horizontal de 107 px sur Pixel 7	TS-07	ENF-05	Ouverte
ANO-06	**Majeure**	29 vulnérabilités connues dans les dépendances frontend (15 hautes, 11 modérées, 3 faibles), dont axios et react-router	TNF-02 (npm audit)	ENF-08	Ouverte
ANO-07	**Mineure**	Dette technique : 55 erreurs ESLint, 10 violations PMD (3 catch vides, equals sans hashCode…), 8 blocs dupliqués (CPD)	TNF-03 (analyse statique)	ENF-06	Ouverte
ANO-08	**Mineure**	Documentation : variables CLOUDINARY_* obligatoires non documentées (le backend ne démarre pas) ; port frontend annoncé 5174 au lieu de 5173	Installation (revue de la documentation)	ENF-07	Ouverte

8.2 Métriques

89 %
exigences couvertes (24/27)

28/33
cas de test réussis (statut final)

49 %
instructions backend couvertes

35,7 ms
p(95) sous 20 utilisateurs

Indicateur	Valeur
Cas de test (statut final)	28 succès · 4 échecs · 1 bloqué
Exécutions enregistrées dans Squash TM	36 (2 itérations)
Tests JUnit	52 (itération 1 : 49 réussis, 3 échecs ; itération 2 : 51 réussis, 1 ignoré)
Tests E2E Playwright	7 (6 réussis, 1 échec)
Étapes manuelles d'acceptation	14 conformes sur 14 (TA-01 à TA-05) + TA-06 en échec

Indicateur	Valeur
Anomalies	8 (1 critique, 4 majeures, 3 mineures) ; 2 corrigées
Efficacité des techniques statiques	5 anomalies sur 8 trouvées sans exécuter le code (revue de code, PMD/ESLint, npm audit, documentation)

8.3 Évaluation des critères de sortie

Critère	Résultat	Verdict
100 % des cas planifiés exécutés	32 exécutés + 1 bloqué (anomalie connue)	**Atteint**
Aucune anomalie critique ouverte	ANO-01 corrigée et re-testée	**Atteint**
Couverture des exigences ≥ 85 %	89 %	**Atteint**
Couverture des instructions ≥ 40 %	49 %	**Atteint**

> **Verdict : recette prononcée AVEC RÉSERVES.** Les critères de sortie sont atteints, mais quatre anomalies majeures restent ouvertes (ANO-03 à ANO-06). Elles doivent être corrigées avant une mise en production.

8.4 Recommandations

Anomalie	Correctif proposé
ANO-03	Dans `AuthController`, remettre `failedLoginAttempts` à 0 lorsque `lockoutUntil` est dépassé, avant de compter le nouvel échec
ANO-04	N'insérer le calendrier initial que si la table est vide : `if (eventRepository.count() == 0) seedEvents();`
ANO-05	Barre latérale repliable (menu « burger ») sous le point de rupture `md` de Tailwind
ANO-06	`npm audit fix`, puis mise à jour d'axios et de react-router ; ajouter l'audit à l'intégration continue
ANO-07	Corriger les erreurs ESLint et PMD ; bloquer la fusion si le lint échoue
ANO-08	Documenter `CLOUDINARY_*` dans le README et `backend/.env.example`
Général	Exécuter `mvn test` et Playwright à chaque pull request (GitHub Actions) pour automatiser la non-régression

9. Conclusion

Ces ateliers ont permis de mener une démarche de test complète sur une application réelle : de l'expression des besoins jusqu'au verdict de recette. **Squash TM** a centralisé les exigences, les cas de test et les campagnes. La matrice de traçabilité montre quelles exigences sont vérifiées, et par quels tests.

La combinaison des techniques s'est avérée efficace. La **revue de code** (statique) a trouvé les défauts les plus graves avant même l'exécution, et les **tests dynamiques** les ont confirmés et ont mesuré leur impact. Les tests **automatisés** (JUnit, Playwright, k6) rendent la non-régression peu coûteuse. Les tests **manuels** restent indispensables pour juger l'expérience utilisateur et explorer des comportements inattendus, comme la perte des événements au redémarrage. Enfin, le cycle **correction → re-test → non-régression** a été appliqué à l'anomalie critique de confidentialité, désormais corrigée.

10. Annexe : rejouer les tests

```
# 1. Infrastructure (Docker)
docker run -d --name enigov-postgres -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=enigov -p 5433:5432
postgres:16
docker exec enigov-postgres psql -U postgres -c "CREATE DATABASE enigov_test;" -c "CREATE DATABASE
squashtm;"
docker network create qa-net && docker network connect qa-net enigov-postgres
docker run -d --name squash-tm --network qa-net -p 8090:8080 -e SQTM_DB_TYPE=postgresql -e
SQTM_DB_HOST=enigov-postgres \
  -e SQTM_DB_NAME=squashtm -e SQTM_DB_USERNAME=postgres -e SQTM_DB_PASSWORD=postgres \
  -e SQUASH_RESTAPI_DISALLOWBASICAUTHENTICATION=false squashtest/squash      # admin / admin

# 2. Application
cd backend  && mvn spring-boot:run          # http://localhost:8081  (backend/.env)
cd frontend && npm install && npm run dev    # http://localhost:5173

# 3. Tests
cd backend && mvn test                       # unitaires + intégration + rapport JaCoCo
(target/site/jacoco)
cd backend && mvn pmd:pmd pmd:cpd            # analyse statique Java
cd frontend && npx eslint . --ext js,jsx     # analyse statique JavaScript
npm audit --workspace=frontend               # dépendances vulnérables
cd qa && npm install && npx playwright install chromium
cd qa && npx playwright test -c e2e/playwright.config.js        # tests système E2E
cd qa && node acceptance/run-acceptance.mjs                     # aide à l'exécution des tests
d'acceptation
docker run --rm -v ./qa/perf:/scripts grafana/k6 run /scripts/charge-api.js   # charge
cd qa && node squash/seed.mjs                                   # alimentation de Squash TM
```

Fichier	Contenu
`backend/src/test/java/com/unigov/unit/`	Tests unitaires (TU-01 à TU-04)
`backend/src/test/java/com/unigov/integration/`	Tests d'intégration (TI-01 à TI-09)
`backend/src/test/resources/application-test.properties`	Profil de test (base enigov_test)
`qa/e2e/`	Tests système Playwright (TS-01 à TS-07)
`qa/acceptance/run-acceptance.mjs`	Enregistrement des preuves des tests d'acceptation
`qa/perf/charge-api.js`	Scénario de charge k6
`qa/squash/`	Référentiel de test et script d'alimentation de Squash TM
`qa/results/`	Résultats bruts, rapports HTML et captures d'écran